

● THE DBA ADMIN / PRACTITIONER SERIES

A FIELD MANUAL FOR DATABASE ENGINEERS

Terraform for the PostgreSQL DBA

Provisioning, hardening and operating PostgreSQL on AWS, GCP, Azure and bare cloud VMs.

The practitioner's bible — nine chapters, five capstone projects, one curated index.

Versha Jain · The DBA Admin

Preface

This book is written for the database engineer who has spent a career tuning autovacuum, debugging replication lag, and writing shell scripts to roll out roles — and who now finds that the platform team has handed them a Terraform repository and a pull-request workflow. It is also written for the platform engineer who knows Terraform deeply but has never had to argue with a DBA about `shared_buffers`.

Terraform is no longer optional for PostgreSQL operators in 2026. Every cloud provider exposes a managed PostgreSQL service whose primary interface is an API, not a console; every regulated environment expects the database topology to be reviewable in a pull request; every on-call rotation expects that a rebuilt replica looks bit-identical to the one that was destroyed. The DBA who treats Terraform as *somebody else's problem* hands away the leverage that makes them effective.

What this book does **not** do is teach you PostgreSQL. We assume you can read EXPLAIN plans, know the difference between a hash and merge join, and have opinions about `max_connections`. We meet you where you are and translate your existing knowledge into the declarative, version-controlled idioms that the rest of your platform now speaks.

How this book is organised

Chapters 1–3 establish the foundation: why Terraform belongs in the DBA toolbox, the parts of HCL you actually need, and the PostgreSQL provider that lets Terraform manage objects *inside* the database. **Chapters 4–7** walk through each major deployment target: self-hosted PostgreSQL on cloud VMs, AWS RDS & Aurora, Google Cloud SQL & AlloyDB, and Azure Database for PostgreSQL Flexible Server. **Chapters 8–9** cover state, CI/CD, security, compliance and day-2 operations — the topics that distinguish a working demo from a production service.

The book concludes with five hands-on capstone projects that combine the techniques into deliverable artefacts: an encrypted RDS instance, a highly-available self-hosted pair with Patroni, a point-in-time-recovery pipeline on Cloud SQL, an RBAC and secret-rotation setup on Azure, and a GitOps schema-migration pipeline.

Conventions used in this book

- HCL examples are written for Terraform 1.7+ and use the public provider names (`hashicorp/aws`, `cyrilgdn/postgresql`, etc.) with version pins.
- Cloud-specific examples target current generally-available services as of May 2026; deprecated resource types (Azure Single Server, RDS classic snapshots) are flagged where relevant.
- Inline commands are formatted in monospace. Files and directories are *italicised*. Callouts in tinted boxes highlight production gotchas worth re-reading at 3 a.m.

Callout convention. Boxed paragraphs flag a non-obvious cost, an irreversible operation, or a behaviour that differs from the official documentation.

Table of Contents

Preface	2
How this book is organised	2
Conventions used in this book	2
Table of Contents	3
Chapter 1 · Why Terraform for the PostgreSQL DBA	7
1.1 The gap that Terraform fills	7
1.2 What ‘DBA as code’ actually covers	7
1.3 The mental model: declarative state	8
1.4 Where Terraform competes and complements	8
1.5 Prerequisites and how to read this book	9
Chapter 2 · Terraform Fundamentals for Database Engineers	10
2.1 The HCL surface you must internalise	10
2.2 Providers, resources, and data sources	11
2.3 Variables, locals, outputs, and the for_each pattern	11
2.4 State and why DBAs should care	12
2.5 Modules	12
2.6 Workspaces and environment separation	13
2.7 The plan/apply lifecycle in practice	13
Chapter 3 · The PostgreSQL Provider — Managing Databases, Roles, and Grants	14
3.1 Configuring the provider	14
3.2 Databases, schemas, and ownership	15
3.3 Roles and password lifecycle	15
3.4 Privileges and the GRANT problem	16
3.5 Extensions	17
3.6 Provider quirks worth knowing	17
Chapter 4 · Self-Hosted PostgreSQL on Cloud VMs	18
4.1 Reference architecture	18
4.2 Terraform layout for a self-hosted topology	19
4.3 The compute resource	19
4.4 Bootstrap with cloud-init	21

4.5 Synchronous replication and failover	21
4.6 Backups with pgBackRest	22
4.7 PgBouncer sidecar	23
Chapter 5 · AWS RDS and Aurora PostgreSQL	24
5.1 RDS for PostgreSQL: aws_db_instance	24
5.2 Aurora PostgreSQL: cluster + instance	25
5.3 RDS Proxy	26
5.4 IAM database authentication	27
5.5 Blue/green deployments and version upgrades	27
Chapter 6 · Google Cloud SQL and AlloyDB for PostgreSQL	28
6.1 Cloud SQL: google_sql_database_instance	29
6.2 The private IP problem: PSA vs PSC	30
6.3 Cloud SQL Auth Proxy	30
6.4 IAM authentication on Cloud SQL	31
6.5 AlloyDB	31
Chapter 7 · Azure Database for PostgreSQL	33
7.1 Flexible Server: azurerm_postgresql_flexible_server	33
7.2 The delegated subnet	34
7.3 Microsoft Entra ID authentication	34
7.4 Server parameters	34
7.5 Read replicas	35
Chapter 8 · State, Workspaces, and CI/CD for DBAs	36
8.1 Remote backends	36
8.2 State surgery	36
8.3 Drift detection	37
8.4 CI/CD pipelines	37
8.5 Higher-level orchestrators	38
Chapter 9 · Security, Compliance, and Day-2 Operations	39
9.1 Secret management	39
9.2 Network surface minimisation	39
9.3 Encryption at rest	39
9.4 Audit logging	40
9.5 Backup verification	40

9.6 Compliance annotations	40
9.7 Cost controls	40
9.8 Day-2 runbooks the DBA should own	41
Part II · Capstone Projects	42
Project 1 · Encrypted RDS with Roles and Backups	43
Objective	43
Prerequisites	43
Architecture	43
Step 1 — Repository layout	43
Step 2 — Substrate	44
Step 3 — The RDS instance	45
Step 4 — In-database resources	46
Step 5 — Apply	47
Step 6 — Validation	47
Extensions	47
Project 2 · HA Self-Hosted Pair with Patroni	48
Objective	48
Prerequisites	48
Architecture	48
Step 1 — Module skeleton	48
Step 2 — etcd cluster	49
Step 3 — PostgreSQL nodes	50
Step 4 — NLB and leader-following target group	51
Step 5 — Validation	51
Cleanup	51
Project 3 · Point-in-Time Recovery Pipeline (Cloud SQL)	52
Objective	52
Prerequisites	52
Architecture	52
Step 1 — Production instance	53
Step 2 — Restore workflow	54
Step 3 — The verification function	54
Step 4 — Validation	55

Project 4 · RBAC and Secret Rotation on Azure Flexible Server	56
Objective	56
Architecture	56
Step 1 — Network and server	57
Step 2 — The four-role model	58
Step 3 — Secrets in Key Vault	59
Step 4 — The rotation function	59
Validation	59
Project 5 · GitOps Schema Migrations with Atlas	60
Objective	60
Why not Terraform for schema?	60
Repository layout	60
Step 1 — The Atlas config	61
Step 2 — The Atlas pipeline	62
Step 3 — The migrator role	62
Step 4 — The schema file	63
Validation	63
Appendix A · Glossary	65
Appendix B · Command Cheat Sheet	66
Index	67

Chapter 1 · Why Terraform for the PostgreSQL DBA

Database administration has always had two halves. One half is the craft of the database itself: query planning, replication, lock contention, vacuum scheduling, the dark art of when to REINDEX. The other half is everything that surrounds it — the storage that the WAL sits on, the network that the standby reaches across, the IAM policy that lets the backup job write to object storage, the firewall rule that lets the application pod connect at all. For most of the last twenty years, those two halves were owned by different teams using different tools. The DBA owned SQL and `postgresql.conf`. Somebody else — sysadmins, then SREs, now platform engineers — owned the substrate.

Terraform collapses that division. When the substrate is declarative, the DBA can no longer credibly point at it and say *not my problem*. More importantly, the DBA who learns to express the substrate declaratively gains something they have never had before: reproducible lower environments, code review for production changes, and audit trails for the questions that compliance teams ask three times a year.

1.1 The gap that Terraform fills

Consider a representative incident. A read replica falls over at 02:14. The on-call DBA logs in, finds that the underlying EBS volume hit its burst-credit floor, increases the volume size, waits twenty minutes for the optimiser to settle, and goes back to bed. The next morning, the change exists in three places: the AWS console (the new volume), the DBA's shell history (the resize command), and a Slack message marked `#dba-changes`. It does not exist in the Terraform module that built the replica. Three months later, somebody runs `terraform apply` during a routine version bump and the volume shrinks back to its original size. The replica fails over to the primary and the same engineer is paged at 04:00.

This is the canonical Terraform value proposition stated from the DBA's side of the table. The cure is not to forbid emergency console changes — sometimes the only correct action at 02:14 is to click the button — but to ensure that every change ends up reflected back in code. The concrete techniques (`terraform import`, `terraform state mv`, drift detectors, post-incident reconciliation) are covered in Chapter 8. The cultural shift is what matters at this point: the database topology is part of the application's source code, and it lives in the same review process as the application's tests.

1.2 What 'DBA as code' actually covers

It is useful to distinguish three concentric rings of database configuration that Terraform can address:

- **The instance and its substrate.** The RDS cluster, the parameter group, the subnet group, the security group, the IAM role, the KMS key. These are the resources of the cloud provider; they are managed by `hashicorp/aws`, `hashicorp/google`, or `hashicorp/azurerm`.

- **Objects inside the database.** Databases, schemas, roles, privileges, extensions, publications, foreign servers. These are managed by the community PostgreSQL provider `cyrilgdn/postgresql` and are the subject of Chapter 3.
- **Schema and data.** Tables, indexes, constraints, materialised views, stored procedures. These are *not* well-managed by Terraform alone. Tools like Atlas, Sqitch, Flyway and Liquibase exist precisely because Terraform’s plan-then-apply model does not map cleanly onto schema migrations that need ordering, transactions, and rollback semantics. Project 5 shows how to integrate Terraform with a migration tool without trying to force the latter into HCL.

***Where Terraform stops.** Resist the temptation to manage individual rows of data, runtime queries, or one-shot backfills with Terraform. Every successful Terraform-for-databases practice has a clear handoff to a purpose-built migration tool. The mistake to avoid is using `null_resource` with a `psql -c` provisioner to execute application DDL.*

1.3 The mental model: declarative state

Terraform is a state-reconciliation engine. You describe the world you want; it inspects the world that exists; it produces a plan; you approve; it makes the world match the description. This is foreign to DBAs accustomed to imperative DDL (`ALTER TABLE ... ADD COLUMN`) where you describe the change rather than the end state. The mental shift is the same one experienced when moving from `iptables` rules to a firewall policy: instead of saying *add this rule*, you say *here are all the rules that should exist*.

The implication for daily work is that Terraform must own anything it is asked to manage, end-to-end. If you give Terraform responsibility for the `pg_hba.conf` entries and a human edits the file by hand, the next `terraform apply` will revert the edit. Half-managed resources are the largest single source of pain in the field. The most disciplined teams answer this with a written ownership contract: every resource is either managed by Terraform or it is not, and the contract is visible in the repository’s README.

1.4 Where Terraform competes and complements

The IaC landscape is no longer Terraform-versus-CloudFormation. As of 2026 the realistic alternatives a DBA will meet are:

Tool	Strengths	DBA-relevant weaknesses
Pulumi	General-purpose languages (Python, TypeScript); basic provider ecosystem	Scripted; basic provider ecosystem; no Terraform-like bridge to re
Crossplane	Kubernetes-native; reconciliation loop is core	Requires a Kubernetes cluster; opinionated about composition; c
AWS CDK / CDKTF	Programmatic synthesis to CloudFormation	Adds Terraform steps. The synthesised output is what actually runs a
Cloud-provider consoles	Lowest cognitive load for one-off changes	Zero reproducibility; the source of every drift story in this chapter

Terraform wins for the DBA persona because it is text-first, provider-rich, and increasingly the lingua franca of the platform teams you have to negotiate with. Knowing it well is more useful than knowing any one alternative.

1.5 Prerequisites and how to read this book

- Terraform 1.7 or newer (1.9 recommended for the removed block and moved-block ergonomics).
- Working credentials for at least one of: AWS, Google Cloud, Azure. The self-hosted chapters assume AWS but translate readily.
- A PostgreSQL 14, 15 or 16 mental model. Examples target PG 16 unless stated; PG 17 features are not yet GA on all managed services.
- Comfort with the command line, `psql`, and basic Git.

The book is designed to be read in order — later chapters depend on vocabulary established in earlier ones — but the cloud-specific chapters (4–7) are self-contained. A DBA who lives entirely on AWS can read Chapter 5 and skip 6 and 7 with no loss of continuity.

Chapter 2 · Terraform Fundamentals for Database Engineers

This chapter covers the parts of Terraform a DBA actually needs. It is not a reference manual; the HashiCorp documentation already does that job. It is a tour of the constructs that show up in every database module, framed in language that maps cleanly to concepts you already have.

2.1 The HCL surface you must internalise

HCL2 is a configuration language with five constructs you cannot avoid: the **block**, the **attribute**, the **expression**, the **reference**, and the **meta-argument**. A representative example:

```
terraform {
  required_version = ">= 1.7.0"
  required_providers {
    aws      = { source = "hashicorp/aws",      version = "~> 5.50" }
    postgresql = { source = "cyrilgdn/postgresql", version = "~> 1.22" }
  }
}

variable "environment" {
  type        = string
  description = "Deployment tier: dev, staging, prod"
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be one of dev/staging/prod."
  }
}

locals {
  name_prefix = "pg-${var.environment}"
  common_tags = {
    Environment = var.environment
    Service     = "postgres"
    ManagedBy   = "terraform"
  }
}

resource "aws_db_subnet_group" "this" {
  name       = "${local.name_prefix}-subnets"
  subnet_ids = data.aws_subnets.db.ids
  tags       = local.common_tags
}

output "subnet_group_name" {
  value = aws_db_subnet_group.this.name
}
```

Read top-to-bottom: the `terraform` block declares the engine and provider constraints (always pin); `variable` blocks declare typed inputs with validations (always validate); `locals` compute reusable expressions; `resource` blocks create real things; `data` blocks read existing things; `output` blocks publish values that other modules or operators can consume.

2.2 Providers, resources, and data sources

A **provider** is the plug-in that knows how to talk to one external API. Every cloud has one or more — hashicorp/aws, hashicorp/google, hashicorp/azurerm. The PostgreSQL *protocol* itself has a provider, cyrilgdn/postgresql, that speaks the wire protocol of any reachable PostgreSQL server, managed or self-hosted.

A **resource** describes something Terraform should create and own. A **data source** describes something that already exists and that Terraform should read. The distinction matters: deleting a resource block *destroys* the thing; deleting a data block just stops reading it. DBAs who are nervous about destructive changes should memorise this asymmetry.

Production discipline. Use `prevent_destroy = true` on lifecycle for any resource whose deletion would lose data — the primary instance, the KMS key, the backup vault. This prevents the most common oops commit from ever reaching apply.

```
resource "aws_db_instance" "primary" {
  identifier      = "${local.name_prefix}-primary"
  engine          = "postgres"
  engine_version  = "16.3"
  instance_class  = "db.r6g.xlarge"
  allocated_storage = 200

  lifecycle {
    prevent_destroy      = true
    ignore_changes       = [password] # rotated out-of-band
    create_before_destroy = false
  }
}
```

2.3 Variables, locals, outputs, and the for_each pattern

Variables are inputs; locals are computed values; outputs are exposed results. The single most useful pattern in any production module is `for_each` over a typed map — it makes N copies of a resource without resorting to imperative loops. DBAs use it to declare a fleet of read replicas, a set of pgBouncer userlists, or a collection of managed databases inside one instance:

```
variable "databases" {
  type = map(object({
    owner      = string
    encoding    = optional(string, "UTF8")
    collate     = optional(string, "en_US.utf8")
  }))
  default = {
    orders = { owner = "orders_app" }
    catalog = { owner = "catalog_app" }
    audit   = { owner = "audit_writer", encoding = "UTF8" }
  }
}

resource "postgresql_database" "this" {
  for_each = var.databases
  name      = each.key
  owner     = each.value.owner
  encoding  = each.value.encoding
  lc_collate = each.value.collate
}
```

The pattern's strength is that the resource *address* becomes `postgresql_database.this["orders"]`, which is stable across additions and removals. Compare with `count`, where inserting a database in the middle of the list re-indexes everything downstream and causes needless recreations. **Use** `for_each`, **not** `count`, for any collection a DBA will edit by hand.

2.4 State and why DBAs should care

Terraform stores its view of the world in a file called the *state*. The state is the source of truth for `terraform plan`: it records that resource `aws_db_instance.primary` maps to `arn:aws:rds:eu-west-1:...:db:pg-prod-primary`. Without state, Terraform would have to re-discover every resource on every run; with state, plans complete in seconds.

State has three properties that matter to a DBA:

- **It contains secrets.** Database passwords, KMS key material, and connection URIs are visible to anyone who can read the state file. Use a remote backend with encryption at rest and IAM-controlled access. Never commit state to Git.
- **It can drift.** Out-of-band changes (a console click, an automation that bypasses Terraform) cause the state to disagree with reality. Drift detection — running `terraform plan` on a schedule and alerting on non-empty plans — is the antidote.
- **It can be surgically edited.** `terraform state mv`, `terraform import`, and the modern moved and removed blocks let you refactor a module without recreating real resources. Chapter 8 covers state surgery in depth.

2.5 Modules

A **module** is a reusable directory of Terraform code. Modules let you encapsulate the noise — subnet groups, parameter groups, IAM policies — and expose a clean interface. The convention used throughout this book is:

```
modules/
  rds-postgres/
    main.tf      # resources
    variables.tf # inputs
    outputs.tf   # outputs
    versions.tf  # provider/version pins
  cloudsql-postgres/
    ...
  azure-flex/
    ...

envs/
  dev/
    main.tf      # consumes modules with dev-flavoured inputs
    backend.tf
  prod/
    main.tf
    backend.tf
```

Pin module versions when sourcing from a registry; if you reference a Git tag, treat it as immutable. The single biggest mistake teams make with modules is consuming a moving main branch and

being surprised when a refactor in the module breaks every environment overnight.

2.6 Workspaces and environment separation

Terraform workspaces are a thin namespacing mechanism on top of one state file. They are **not** the right tool for separating prod from non-prod — the production state should live in a completely different backend, with different credentials, ideally in a different cloud account. Use workspaces for cheap one-off variants (per-developer scratch environments, ephemeral PR previews); use directories or whole repositories for tiered environments.

2.7 The plan/apply lifecycle in practice

A correct DBA workflow looks like the following:

- `terraform fmt` — canonical formatting (enforced in CI).
- `terraform validate` — syntactic and schema checks.
- `terraform plan -out=tfplan` — produce a binary plan; review it with `terraform show tfplan` or post-process with `jq`.
- Peer review of the plan output (and the diff) in a pull request.
- `terraform apply tfplan` — apply the **same** plan that was reviewed. Never re-plan during apply.

***Read the plan literally.** The line `~ allocated_storage: 200 -> 100` means Terraform is about to shrink your storage by 50%. Plans are unambiguous; misread plans are a recurring root-cause in post-mortems. Train yourself to spot `-/+` (destroy and recreate) on stateful resources.*

Chapter 3 · The PostgreSQL Provider — Managing Databases, Roles, and Grants

The cloud providers stop at the instance boundary. They will create an RDS cluster, an AlloyDB instance, a Flexible Server — but the databases, roles, schemas, extensions and privileges inside that instance are out of scope. To manage *inside* the PostgreSQL process, you use the community provider `cyrilgdn/postgresql`. Every chapter that follows assumes you are comfortable with this provider; this chapter is where that comfort is established.

3.1 Configuring the provider

The provider needs network reachability and credentials. In a well-designed deployment, the credentials come from a secret manager and never appear in HCL. For development, you can read them from environment variables; for production, fetch them with a data source:

```
provider "postgresql" {
  host          = aws_db_instance.primary.address
  port          = aws_db_instance.primary.port
  database      = "postgres"
  username      = "tf_admin"
  password      = data.aws_secretsmanager_secret_version.tf_admin.secret_string
  superuser     = false
  sslmode       = "require"
  connect_timeout = 15
}
```

Three configuration points trip up newcomers. First, set `superuser = false` on managed services — RDS, Cloud SQL and Azure Flexible Server *do not* grant the SUPERUSER attribute, and leaving the default in place will cause the provider to attempt operations that fail with permission errors. Second, use `sslmode = "require"` at minimum — `disable` is a footgun, and `verify-full` is the gold standard once you have wired up a trusted CA. Third, the provider speaks the standard libpq protocol, so it works equally well against every PostgreSQL service in this book.

3.2 Databases, schemas, and ownership

```
resource "postgresql_role" "orders_app" {
  name      = "orders_app"
  login     = true
  password  = random_password.orders_app.result
  encrypted_password = true
}

resource "postgresql_database" "orders" {
  name            = "orders"
  owner           = postgresql_role.orders_app.name
  encoding        = "UTF8"
  lc_collate      = "en_US.utf8"
  lc_ctype        = "en_US.utf8"
  template        = "template0"
  connection_limit = 200
  allow_connections = true
}

resource "postgresql_schema" "ledger" {
  name      = "ledger"
  database = postgresql_database.orders.name
  owner     = postgresql_role.orders_app.name
}
```

Two patterns are worth internalising. First, the *owner* of an object is the role that can drop it. Owning databases as application roles — rather than as a shared admin — means a misconfigured pipeline cannot drop a sibling database. Second, always create databases from `template0`, not `template1`: `template1` can be modified, and inheriting its state at provisioning time has caused production incidents involving rogue extensions.

3.3 Roles and password lifecycle

Managing PostgreSQL roles with Terraform requires resolving a tension: Terraform wants the password in state; security wants the password ephemeral. The most common reconciliation is to generate the password with `random_password`, write it to a secret manager, and tell Terraform to ignore subsequent drift on the password attribute so that out-of-band rotations do not get reverted:

```
resource "random_password" "orders_app" {
  length      = 32
  special     = true
  override_special = " !#$%&*()_-+[]{}<&gt;:?"
  keepers = {
    rotation_id = var.rotation_id
  }
}

resource "aws_secretsmanager_secret" "orders_app" {
  name      = "${local.name_prefix}/orders_app"
  description = "Application role for the orders DB"
  kms_key_id = aws_kms_key.db_secrets.id
}

resource "aws_secretsmanager_secret_version" "orders_app" {
  secret_id      = aws_secretsmanager_secret.orders_app.id
  secret_string = jsonencode({
```

```

    username = "orders_app"
    password = random_password.orders_app.result
    host      = aws_db_instance.primary.address
    port      = aws_db_instance.primary.port
    dbname    = "orders"
  })
}

resource "postgresql_role" "orders_app" {
  name           = "orders_app"
  login          = true
  password       = random_password.orders_app.result
  encrypted_password = true
  connection_limit = 100
  valid_until    = "infinity"

  lifecycle { ignore_changes = [password] }
}

```

The keepers map on `random_password` is the rotation handle: bumping `var.rotation_id` regenerates the password on the next apply, which Terraform writes to the secret. Applications consume the secret at boot, so a rotation is decoupled from deployment timing. The `ignore_changes` on password means a manual rotation through `ALTER ROLE` by an on-call DBA will not be reverted.

3.4 Privileges and the GRANT problem

Privileges in PostgreSQL are notoriously stateful: a `GRANT` applies to objects that *exist at the time of the grant*, and `ALTER DEFAULT PRIVILEGES` applies to objects created *after* the alter. The provider exposes both:

```

resource "postgresql_grant" "orders_app_dml" {
  database      = postgresql_database.orders.name
  role          = postgresql_role.orders_app.name
  schema        = postgresql_schema.ledger.name
  object_type   = "table"
  privileges    = ["SELECT", "INSERT", "UPDATE", "DELETE"]
}

resource "postgresql_default_privileges" "orders_app_future_tables" {
  database      = postgresql_database.orders.name
  role          = postgresql_role.orders_app.name
  schema        = postgresql_schema.ledger.name
  owner         = postgresql_role.orders_app.name
  object_type   = "table"
  privileges    = ["SELECT", "INSERT", "UPDATE", "DELETE"]
}

```

Always pair the two. A `postgresql_grant` without a corresponding `postgresql_default_privileges` is a permission gap waiting to bite on the next migration. The owner argument on the latter is the role that will create future objects — usually the application role itself for self-owned schemas, or a dedicated migrator role for centralised migration tools.

3.5 Extensions

Extensions are first-class. On managed services you can install only those on the provider's allow-list (see `aws_rds_engine_version.supported_feature_names` on RDS, `cloudsql.enable_pgaudit` on Cloud SQL, etc.), but the HCL is identical:

```
resource "postgresql_extension" "pg_stat_statements" {
  name      = "pg_stat_statements"
  database  = postgresql_database.orders.name
  schema    = "public"
}

resource "postgresql_extension" "pgcrypto" {
  name      = "pgcrypto"
  database  = postgresql_database.orders.name
}
```

Some extensions require a server restart or a parameter group change first; the parameter group must be applied *before* the extension is created, which means an explicit `depends_on` is sometimes necessary.

3.6 Provider quirks worth knowing

- The provider opens one connection per parallel operation; the default parallelism of 10 can saturate a small instance during a fresh apply. Pass `-parallelism=4` for initial bootstraps.
- `postgresql_role` with `encrypted_password = true` stores the SCRAM-SHA-256 hash; `false` stores plaintext — never use the latter.
- Cross-database GRANTS require one provider instance per database. Use provider aliases when an instance hosts more than one logical tenant.
- Destroying a `postgresql_database` resource issues `DROP DATABASE`. Set `lifecycle.prevent_destroy = true` on production databases.

Bootstrap chicken-and-egg. The provider needs to connect to the database before it can manage it, but the database may not yet exist on the first apply. The pattern is to split bootstrap into two Terraform invocations or to use `-target`: first apply the cloud resources (RDS, Cloud SQL, ...), then apply the in-database resources with the provider pointed at the freshly-created endpoint.

Chapter 4 · Self-Hosted PostgreSQL on Cloud VMs

Managed databases are convenient and, for many teams, correct. But the DBA market reality is that a non-trivial fraction of PostgreSQL workloads still run on raw cloud VMs — for cost, for control over extensions, for licensing, for compliance regimes that forbid third-party operators, or for sheer scale that exceeds what a managed service tier will give you. This chapter shows how to manage a self-hosted topology with Terraform without giving up the operational ergonomics that managed services provide for free.

4.1 Reference architecture

The pattern in this chapter is deliberately conservative: two EC2 instances (primary and synchronous standby) in different availability zones, each with a dedicated EBS volume for data and a separate volume for WAL, fronted by a single Network Load Balancer that resolves to the current primary. Patroni handles leader election against a three-node etcd cluster. pgBackRest performs continuous WAL archiving and full/diff/incremental backups to S3.

Every cloud has analogues. On GCP use Compute Engine + persistent disks + Cloud Storage. On Azure use Virtual Machines + managed disks + Blob Storage. The Terraform shape — networking, compute, storage, bootstrap — is identical.

4.2 Terraform layout for a self-hosted topology

```
modules/
  vpc-database/      # VPC, subnets, route tables, NACLs
  postgres-vm/       # one PG node: EC2 + EBS + IAM
  patroni-cluster/   # composes N postgres-vm + etcd + NLB
  pgbackrest-repo/   # S3 bucket + lifecycle + IAM

envs/prod/main.tf:

module "vpc" { source = "../../modules/vpc-database" ... }

module "etcd" {
  source      = "../../modules/etcd-cluster"
  subnet_ids = module.vpc.private_subnet_ids
  count_dcs   = 3
}

module "pg_cluster" {
  source          = "../../modules/patroni-cluster"
  cluster_name    = "pg-prod"
  pg_version      = "16"
  instance_type   = "r6i.2xlarge"
  data_volume_gb  = 1000
  wal_volume_gb   = 200
  primary_az      = "eu-west-1a"
  standby_az      = "eu-west-1b"
  etcd_endpoints  = module.etcd.endpoints
  backup_bucket   = module.backup.bucket_name
  subnet_ids      = module.vpc.private_subnet_ids
  security_groups = [module.vpc.db_security_group_id]
}
```

4.3 The compute resource

A production PostgreSQL VM differs from a default EC2 instance in several ways the Terraform code must reflect:

- **Storage is the workload.** A single root volume is wrong. Mount `/var/lib/postgresql` on a dedicated gp3 or io2 volume sized for throughput, not IOPS; mount `pg_wal` on a second, smaller, high-throughput volume so WAL fsyncs do not compete with checkpoint writes.
- **Instance types matter.** Prefer the `r` family for memory headroom; avoid burst-class (`t` family) for any production database because the CPU credit floor is a hidden P95 spike waiting to happen.
- **EBS-optimised and Nitro.** Modern instance generations are EBS-optimised by default, but verify; older types throttle EBS through the same pipe as the network and you will see it during backups.
- **Tenancy and placement.** Use a placement group with cluster strategy for primary and standby to minimise inter-AZ latency on synchronous replication.

```
resource "aws_instance" "pg_node" {
  for_each = { for n in var.nodes : n.name => n }

  ami          = data.aws_ami.ubuntu_22.id
  instance_type = var.instance_type
```

```

subnet_id            = each.value.subnet_id
vpc_security_group_ids = var.security_groups
iam_instance_profile  = aws_iam_instance_profile.pg_node.name
ebs_optimized         = true
monitoring            = true

root_block_device {
  volume_type      = "gp3"
  volume_size      = 50
  encrypted         = true
  kms_key_id        = var.kms_key_arn
  delete_on_termination = true
}

metadata_options {
  http_tokens            = "required" # IMDSv2 only
  http_put_response_hop_limit = 1
}

user_data = templatefile("${path.module}/cloud-init.yaml.tftpl", {
  pg_version      = var.pg_version
  cluster_name    = var.cluster_name
  node_name       = each.key
  etcd_endpoints  = var.etcd_endpoints
  backup_bucket   = var.backup_bucket
})

tags = merge(var.tags, { Role = "postgres", Node = each.key })

lifecycle { ignore_changes = [ami, user_data] }
}

resource "aws_ebs_volume" "pg_data" {
  for_each      = aws_instance.pg_node
  availability_zone = each.value.availability_zone
  size          = var.data_volume_gb
  type          = "gp3"
  iops          = 12000
  throughput    = 500
  encrypted     = true
  kms_key_id    = var.kms_key_arn
  tags          = merge(var.tags, { MountPoint = "/var/lib/postgresql" })
}

resource "aws_volume_attachment" "pg_data" {
  for_each      = aws_ebs_volume.pg_data
  device_name   = "/dev/xvdf"
  volume_id     = each.value.id
  instance_id   = aws_instance.pg_node[each.key].id
}

```

Why `ignore_changes = [ami, user_data]`. Both attributes force instance replacement, which means destroying the data volume attachment. For a self-hosted database that is catastrophic. Patch by rebuilding standby first, promoting, then rebuilding the old primary — do not let Terraform make that decision implicitly.

4.4 Bootstrap with cloud-init

The cloud-init template installs PostgreSQL, Patroni, and pgBackRest, wires Patroni to etcd, and mounts the data volume. The template is deliberately small — everything beyond the bootstrap belongs in a configuration-management tool (Ansible) or in Patroni's `postgresql.parameters` block, which is itself stored in etcd and reloaded without a Terraform run.

```
#cloud-config
package_update: true
packages:
  - postgresql-${pg_version}
  - postgresql-${pg_version}-pgaudit
  - python3-pip
  - etcd-client
  - pgbackrest

runcmd:
  - pip3 install patroni[etcd3] psycopg2-binary
  - mkfs.ext4 -L pgdata /dev/nvme1n1
  - mkdir -p /var/lib/postgresql/${pg_version}/main
  - echo "LABEL=pgdata /var/lib/postgresql/${pg_version}/main ext4 defaults,noatime 0 0" >>> /etc/fstab
  - mount -a
  - chown -R postgres:postgres /var/lib/postgresql
  - install -m 0644 -o postgres -g postgres /dev/stdin /etc/patroni/patroni.yml <<<'YAML'
    scope: ${cluster_name}
    name: ${node_name}
    etcd3:
      hosts: ${etcd_endpoints}
  bootstrap:
    dcs:
      synchronous_mode: true
      postgresql:
        use_pg_rewind: true
        parameters:
          shared_buffers: 8GB
          effective_cache_size: 24GB
          wal_level: replica
          max_wal_senders: 10
          archive_mode: on
          archive_command: 'pgbackrest --stanza=main archive-push %p'
    YAML
  - systemctl enable --now patroni
```

4.5 Synchronous replication and failover

Patroni handles leader election; Terraform handles only the substrate. The Terraform-relevant choices are the *shape* of the cluster (how many nodes, in which AZs) and the *traffic plane* (the NLB target group whose health checks identify the current primary).

```
resource "aws_lb_target_group" "pg_primary" {
  name      = "${var.cluster_name}-primary"
  port      = 5432
  protocol  = "TCP"
  vpc_id    = var.vpc_id
  target_type = "instance"

  health_check {
    protocol = "HTTP"
    port     = "8008"           # Patroni REST API
  }
}
```

```

    path          = "/primary"          # 200 only when this node is leader
    interval      = 5
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}

resource "aws_lb_target_group_attachment" "pg_primary" {
  for_each      = aws_instance.pg_node
  target_group_arn = aws_lb_target_group.pg_primary.arn
  target_id     = each.value.id
}

```

Patroni exposes `/primary`, `/replica`, `/standby-leader` and other endpoints on port 8008. The NLB's health check — *not* Terraform — decides which node serves writes. Terraform merely attaches every node to the target group; routing follows leader election in real time. Failover therefore happens without any Terraform action, which is what you want.

4.6 Backups with pgBackRest

pgBackRest is the production-grade backup tool for self-hosted PostgreSQL. It does WAL archiving, parallel full/diff/incremental backups, point-in-time recovery, encryption, and integrity verification. Terraform manages the repository (an S3 bucket with lifecycle policies) and the IAM role attached to each PG node; pgBackRest itself is invoked by cron or by Patroni hooks on the nodes.

```

resource "aws_s3_bucket" "backup" {
  bucket = "${var.cluster_name}-backups"
}

resource "aws_s3_bucket_versioning" "backup" {
  bucket = aws_s3_bucket.backup.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "backup" {
  bucket = aws_s3_bucket.backup.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "aws:kms"
      kms_master_key_id = var.kms_key_arn
    }
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "backup" {
  bucket = aws_s3_bucket.backup.id
  rule {
    id      = "wal-retention"
    status  = "Enabled"
    filter { prefix = "archive/" }
    expiration { days = 35 } # 5 weeks of WAL
  }
  rule {
    id      = "backup-retention"
    status  = "Enabled"
    filter { prefix = "backup/" }
    transition {

```

```
    days          = 30
    storage_class = "GLACIER_IR"
  }
  expiration { days = 365 }
}
```

4.7 PgBouncer sidecar

A self-hosted PostgreSQL fronted by application connection pools is fine until the application fleet's pool count exceeds `max_connections`. The remedy is a transaction-mode pooler — PgBouncer — deployed close to the database (on the same VM or as a co-located service). The Terraform footprint is a `aws_ecs_service` or a second user-data script; the configuration footprint — `userlist.txt` and `pgbouncer.ini` — is templated from the same role list that `cyrilgdn/postgresql` manages, which gives you a single source of truth.

Self-hosted is a relationship, not a project. A managed service's minor version bump is a one-line change; a self-hosted minor version bump is a maintenance window. Budget the operational cost honestly — in head-count, not just in dollars saved.

Chapter 5 · AWS RDS and Aurora PostgreSQL

AWS exposes three PostgreSQL services that overlap in confusing ways: RDS for PostgreSQL (a managed single-engine instance), Aurora PostgreSQL (a re-architected storage layer that speaks PostgreSQL), and Aurora Serverless v2 (Aurora with auto-scaling compute). This chapter walks through each in Terraform and ends with the cross-cutting concerns — RDS Proxy, IAM authentication, parameter groups, blue/green deployments — that apply to all three.

5.1 RDS for PostgreSQL: `aws_db_instance`

RDS is the right default for workloads under roughly 10 TB and fewer than 100k transactions per second — the threshold above which Aurora's storage architecture starts to pay for itself. The canonical Terraform shape is one parameter group, one option group, one subnet group, and one instance, plus a Multi-AZ standby for production:

```
resource "aws_db_parameter_group" "pg16" {
  name       = "${local.name_prefix}-pg16"
  family     = "postgres16"
  description = "Tuned parameter group for ${local.name_prefix}"

  parameter { name = "shared_preload_libraries" value = "pg_stat_statements,pgaudit" apply_method = "pending-reboot" }
  parameter { name = "log_min_duration_statement" value = "1000" }
  parameter { name = "log_lock_waits" value = "1" }
  parameter { name = "log_statement" value = "ddl" }
  parameter { name = "pgaudit.log" value = "ddl, role, write" }
  parameter { name = "rds.force_ssl" value = "1" apply_method = "pending-reboot" }
}

resource "aws_db_subnet_group" "this" {
  name       = "${local.name_prefix}-subnets"
  subnet_ids = data.aws_subnets.db.ids
}

resource "aws_db_instance" "primary" {
  identifier      = "${local.name_prefix}-primary"
  engine          = "postgres"
  engine_version  = "16.3"
  instance_class  = "db.r6g.2xlarge"
  allocated_storage = 500
  max_allocated_storage = 2000          # storage autoscaling
  storage_type     = "gp3"
  storage_throughput = 500
  iops             = 12000
  storage_encrypted = true
  kms_key_id       = aws_kms_key.rds.arn

  db_name      = "appdb"
  username     = "rds_admin"
  manage_master_user_password = true      # AWS-managed secret in Secrets Manager
  master_user_secret_kms_key_id = aws_kms_key.rds.arn

  db_subnet_group_name = aws_db_subnet_group.this.name
}
```

```

vpc_security_group_ids = [aws_security_group.db.id]
parameter_group_name   = aws_db_parameter_group.pg16.name

multi_az                = true
publicly_accessible    = false
port                    = 5432

backup_retention_period = 35
backup_window           = "02:00-03:00"
copy_tags_to_snapshot  = true
deletion_protection     = true
delete_automated_backups = false
skip_final_snapshot     = false
final_snapshot_identifier = "${local.name_prefix}-final"

performance_insights_enabled          = true
performance_insights_retention_period = 731      # two years
monitoring_interval                   = 60
monitoring_role_arn                   = aws_iam_role.rds_enhanced_monitoring.arn
enabled_cloudwatch_logs_exports       = ["postgresql", "upgrade"]

auto_minor_version_upgrade = false
apply_immediately          = false

iam_database_authentication_enabled = true
blue_green_update { enabled = true }

tags = local.common_tags

lifecycle {
  prevent_destroy = true
  ignore_changes = [engine_version]    # promoted by maintenance windows
}
}

```

Two attributes are non-obvious. `manage_master_user_password` hands the master credential to AWS Secrets Manager — you never see it, and rotation is managed by AWS on a schedule you configure. `blue_green_update.enabled` instructs RDS to perform major version upgrades using a blue/green deployment behind the scenes; without it, *any* major version change is an outage.

5.2 Aurora PostgreSQL: cluster + instance

Aurora separates storage from compute. The `aws_rds_cluster` resource owns the storage; `aws_rds_cluster_instance` resources own the compute. A typical Aurora topology is one writer plus 1–15 readers:

```

resource "aws_rds_cluster" "aurora" {
  cluster_identifier = "${local.name_prefix}-aurora"
  engine             = "aurora-postgresql"
  engine_version     = "16.2"
  engine_mode        = "provisioned"
  database_name      = "appdb"
  master_username    = "aurora_admin"
  manage_master_user_password = true
  master_user_secret_kms_key_id = aws_kms_key.rds.arn

  db_subnet_group_name = aws_db_subnet_group.this.name
  vpc_security_group_ids = [aws_security_group.db.id]
}

```

```

storage_encrypted      = true
kms_key_id             = aws_kms_key.rds.arn
backup_retention_period = 35
preferred_backup_window = "02:00-03:00"
deletion_protection    = true
copy_tags_to_snapshot  = true

enabled_cloudwatch_logs_exports = ["postgresql"]

serverlessv2_scaling_configuration {
  min_capacity = 0.5
  max_capacity = 16
}

iam_database_authentication_enabled = true

tags = local.common_tags
lifecycle { prevent_destroy = true }
}

resource "aws_rds_cluster_instance" "writer" {
  identifier      = "${local.name_prefix}-aurora-writer"
  cluster_identifier = aws_rds_cluster.aurora.id
  instance_class  = "db.serverless"
  engine          = aws_rds_cluster.aurora.engine
  engine_version  = aws_rds_cluster.aurora.engine_version
  db_parameter_group_name = aws_rds_cluster_parameter_group.aurora_pg16.name
  performance_insights_enabled = true
}

resource "aws_rds_cluster_instance" "readers" {
  for_each = toset(["a", "b"])
  identifier = "${local.name_prefix}-aurora-reader-${each.key}"
  cluster_identifier = aws_rds_cluster.aurora.id
  instance_class  = "db.serverless"
  engine          = aws_rds_cluster.aurora.engine
  engine_version  = aws_rds_cluster.aurora.engine_version
  performance_insights_enabled = true
}

```

Aurora's endpoints are derived attributes: `aws_rds_cluster.aurora.endpoint` is the writer; `aws_rds_cluster.aurora.reader_endpoint` is the load-balanced reader. *Always* connect applications to the right endpoint; a writer endpoint to a read-only workload wastes the entire point of Aurora.

5.3 RDS Proxy

RDS Proxy is a transaction-mode connection pooler that lives inside AWS. It is useful for serverless application tiers (Lambda, Fargate) whose churn would otherwise overwhelm the database. It is *essential* for IAM authentication at scale — the proxy holds the long-lived connections so each invocation pays only the token cost.

```

resource "aws_db_proxy" "app" {
  name           = "${local.name_prefix}-proxy"
  engine_family  = "POSTGRESQL"
  role_arn       = aws_iam_role.proxy.arn
  vpc_subnet_ids = data.aws_subnets.db.ids
  vpc_security_group_ids = [aws_security_group.db.id]
  require_tls    = true
}

```

```

idle_client_timeout    = 1800

auth {
  auth_scheme = "SECRETS"
  iam_auth    = "REQUIRED"
  secret_arn  = aws_secretsmanager_secret.app.arn
}

resource "aws_db_proxy_default_target_group" "app" {
  db_proxy_name = aws_db_proxy.app.name
  connection_pool_config {
    max_connections_percent = 75
    max_idle_connections_percent = 25
    connection_borrow_timeout = 120
  }
}

```

5.4 IAM database authentication

IAM authentication replaces passwords with short-lived tokens that AWS signs. The Terraform footprint is small — `iam_database_authentication_enabled = true` on the instance, plus an IAM policy that allows `rds-db:connect`. The PostgreSQL footprint is one role per IAM principal, with the `rds_iam` group:

```

resource "postgresql_role" "ci_runner" {
  name = "ci_runner"
  login = true
  roles = ["rds_iam"] # AWS-managed group enabling IAM auth
}

resource "aws_iam_policy" "rds_connect_ci" {
  name = "rds-connect-ci"
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = "rds-db:connect"
      Resource = "arn:aws:rds-db:${var.region}:${var.account_id}:dbuser:${aws_db_instance.primary.res
    }]
  })
}

```

5.5 Blue/green deployments and version upgrades

Major version upgrades are the most disruptive routine task in a DBA's year. RDS blue/green automates the painful parts: clone the database, configure logical replication, run the upgrade on the green copy, switch over in a single endpoint flip. Terraform participates by enabling the feature on the instance (shown above) and by orchestrating the cutover via the `aws_rds_export_task` resource if you want a snapshot in between. The actual cutover is initiated via the AWS API or console; Terraform's job is to make sure the substrate supports it.

Parameter group drift. A parameter group attached to a running instance with `apply_method = pending-reboot` shows as 'sync'd' in Terraform even when the instance has not yet rebooted. The change is not effective until the next maintenance window. Track pending-reboot changes out-of-band; Terraform will not alert you.

Chapter 6 · Google Cloud SQL and AlloyDB for PostgreSQL

Google Cloud offers two PostgreSQL services. **Cloud SQL** is the long-established managed PostgreSQL with a feature set comparable to RDS. **AlloyDB** is Google's re-architected, performance-oriented offering, analogous to Aurora's relationship with RDS. This chapter covers both, plus the networking choices — Private Service Access vs. Private Service Connect — that confound newcomers.

6.1 Cloud SQL: google_sql_database_instance

```
resource "google_sql_database_instance" "primary" {
  name                = "${local.name_prefix}-primary"
  database_version    = "POSTGRES_16"
  region             = var.region
  deletion_protection = true

  settings {
    tier                = "db-custom-4-16384"
    availability_type   = "REGIONAL"           # HA across zones
    disk_type           = "PD_SSD"
    disk_size           = 200
    disk_autoresize     = true
    disk_autoresize_limit = 1000

    backup_configuration {
      enabled                = true
      point_in_time_recovery_enabled = true
      start_time              = "02:00"
      transaction_log_retention_days = 7
      backup_retention_settings {
        retained_backups = 30
        retention_unit    = "COUNT"
      }
    }
  }

  ip_configuration {
    ipv4_enabled    = false
    private_network = google_compute_network.vpc.id
    ssl_mode        = "ENCRYPTED_ONLY"
    enable_private_path_for_google_cloud_services = true
  }

  database_flags { name = "log_min_duration_statement" value = "1000" }
  database_flags { name = "log_lock_waits"                value = "on" }
  database_flags { name = "pgaudit.log"                    value = "ddl,role,write" }
  database_flags { name = "cloudsql.enable_pgaudit"        value = "on" }

  insights_config {
    query_insights_enabled = true
    query_string_length    = 4500
    record_application_tags = true
    record_client_address   = true
  }

  maintenance_window {
    day      = 7
    hour     = 3
    update_track = "stable"
  }
}

lifecycle { prevent_destroy = true }
```

Cloud SQL's tier names are coordinate-style: db-custom-**<vCPUs>-<memoryMiB>**. Always use REGIONAL availability for production; the ZONAL default is single-AZ. Always disable `ipv4_enabled` and connect over private IP — a Cloud SQL instance with a public IP is a finding in every audit you will face.

6.2 The private IP problem: PSA vs PSC

Connecting privately to Cloud SQL has two mechanisms and most teams pick the wrong one. **Private Service Access (PSA)** is the long-established mechanism: a VPC peering between your VPC and Google's service producer VPC. It is simple but it consumes a /24 from your address space and the peering is non-transitive. **Private Service Connect (PSC)** is the newer mechanism: a forwarding rule in your VPC that resolves to the Cloud SQL instance. It is more flexible — works across organisations, no peering — and is the right choice for new deployments.

```
# PSA setup
resource "google_compute_global_address" "private_ip_range" {
  name           = "${local.name_prefix}-psa"
  purpose        = "VPC_PEERING"
  address_type   = "INTERNAL"
  prefix_length  = 16
  network        = google_compute_network.vpc.id
}

resource "google_service_networking_connection" "psa" {
  network            = google_compute_network.vpc.id
  service            = "servicenetworking.googleapis.com"
  reserved_peering_ranges = [google_compute_global_address.private_ip_range.name]
}

# Cloud SQL with PSC
resource "google_sql_database_instance" "psc_primary" {
  name             = "${local.name_prefix}-psc"
  database_version = "POSTGRES_16"
  region           = var.region
  settings {
    tier = "db-custom-4-16384"
    ip_configuration {
      ipv4_enabled = false
      psc_config {
        psc_enabled            = true
        allowed_consumer_projects = [var.project_id]
      }
    }
  }
}
```

6.3 Cloud SQL Auth Proxy

The Cloud SQL Auth Proxy is the canonical client. It authenticates with the caller's service-account credentials, opens an mTLS tunnel to the instance, and exposes a local Unix socket or loopback port. Terraform does not manage the proxy directly — it runs as a sidecar in GKE or as a binary on a Compute Engine VM — but Terraform *does* manage the IAM grants:

```
resource "google_project_iam_member" "client_can_connect" {
  project = var.project_id
  role    = "roles/cloudsql.client"
  member  = "serviceAccount:${google_service_account.app.email}"
}

resource "google_project_iam_member" "client_can_iam_auth" {
  project = var.project_id
  role    = "roles/cloudsql.instanceUser"
  member  = "serviceAccount:${google_service_account.app.email}"
}
```

```
}
```

6.4 IAM authentication on Cloud SQL

Cloud SQL supports IAM authentication for both human users (via `iam_authentication = on` database flag) and service accounts. The in-database user name is the email of the principal, with `gserviceaccount.com` stripped for service accounts. The `postgresql_role` resource is still needed — Cloud SQL does not auto-create the role from the IAM grant alone:

```
resource "google_sql_user" "app_sa" {
  name      = trimsuffix(google_service_account.app.email, ".gserviceaccount.com")
  instance  = google_sql_database_instance.primary.name
  type      = "CLOUD_IAM_SERVICE_ACCOUNT"
}
```

6.5 AlloyDB

AlloyDB's resource model differs from Cloud SQL: there is a *cluster*, then one or more *instances* (primary, secondary, or read pool). A typical production setup is one primary instance plus a read pool for analytical traffic:

```
resource "google_alloydb_cluster" "main" {
  cluster_id = "${local.name_prefix}-alloy"
  location   = var.region
  network_config {
    network = google_compute_network.vpc.id
  }
  database_version = "POSTGRES_16"

  initial_user {
    user      = "alloy_admin"
    password = random_password.alloy_admin.result
  }

  automated_backup_policy {
    location      = var.region
    backup_window = "3600s"
    enabled       = true
    weekly_schedule {
      days_of_week = ["MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY", "SATURDAY", "SUNDAY"]
      start_times { hours = 2 minutes = 0 seconds = 0 nanos = 0 }
    }
    quantity_based_retention { count = 14 }
  }

  continuous_backup_config {
    enabled           = true
    recovery_window_days = 14
  }

  deletion_policy = "FORCE" # set to DEFAULT in prod for safety
  lifecycle { prevent_destroy = true }
}

resource "google_alloydb_instance" "primary" {
  cluster      = google_alloydb_cluster.main.name
  instance_id  = "${local.name_prefix}-primary"
  instance_type = "PRIMARY"
```

```

machine_config { cpu_count = 8 }

database_flags = {
  "log_min_duration_statement" = "1000"
  "pgaudit.log"                = "ddl,role,write"
}

resource "google_alloydb_instance" "read_pool" {
  cluster      = google_alloydb_cluster.main.name
  instance_id  = "${local.name_prefix}-readpool"
  instance_type = "READ_POOL"
  read_pool_config { node_count = 3 }

  depends_on = [google_alloydb_instance.primary]
}

```

AlloyDB's read pool is a single logical endpoint that load-balances over N nodes — scaling reads is a `node_count` edit, not a topology change. The columnar engine (enabled by default on 4+ vCPU instances) accelerates analytical queries transparently; the only Terraform-visible knob is the instance size.

Cost discipline on AlloyDB. *The minimum production cluster is noticeably more expensive than the smallest Cloud SQL instance. AlloyDB pays off when the workload exercises its strengths (read-heavy, analytical, large) — not as a drop-in replacement for small OLTP.*

Chapter 7 · Azure Database for PostgreSQL

Azure's PostgreSQL offering has consolidated. The legacy *Single Server* is on a retirement path; new work belongs on **Flexible Server**, which is the focus of this chapter. Flexible Server supports zone-redundant HA, custom maintenance windows, Microsoft Entra ID (formerly Azure AD) authentication, private networking via VNet delegation, and a clean Terraform resource type.

7.1 Flexible Server: `azurerm_postgresql_flexible_server`

```
resource "azurerm_postgresql_flexible_server" "this" {
  name                       = "${local.name_prefix}-flex"
  resource_group_name       = azurerm_resource_group.db.name
  location                  = azurerm_resource_group.db.location
  version                   = "16"
  delegated_subnet_id       = azurerm_subnet.db.id
  private_dns_zone_id       = azurerm_private_dns_zone.flex.id

  administrator_login       = "pg_admin"
  administrator_password    = random_password.pg_admin.result

  sku_name                  = "GP_Standard_D4ds_v5"           # General Purpose, 4 vCore
  storage_mb                = 524288                         # 512 GB
  storage_tier               = "P30"
  zone                      = "1"

  high_availability {
    mode                  = "ZoneRedundant"
    standby_availability_zone = "2"
  }

  backup_retention_days      = 35
  geo_redundant_backup_enabled = true

  authentication {
    active_directory_auth_enabled = true
    password_auth_enabled         = true
    tenant_id                     = data.azurerm_client_config.current.tenant_id
  }

  maintenance_window {
    day_of_week = 0      # Sunday
    start_hour  = 2
    start_minute = 0
  }

  lifecycle {
    prevent_destroy = true
    ignore_changes = [administrator_password, zone]
  }
}
```

Several attributes deserve a second look. `delegated_subnet_id` places the server's NIC in your VNet — this is the only supported pattern for private networking on Flexible Server, and the

subnet must be *delegated* to Microsoft.DBforPostgreSQL/flexibleServers. `ignore_changes = [zone]` protects you against an Azure-initiated failover changing the recorded zone — without this, the next `apply` will plan to move the server back.

7.2 The delegated subnet

```
resource "azurerm_subnet" "db" {
  name                       = "snet-postgres"
  resource_group_name       = azurerm_resource_group.network.name
  virtual_network_name      = azurerm_virtual_network.this.name
  address_prefixes          = ["10.20.16.0/24"]

  service_endpoints = ["Microsoft.Storage"]
  delegation {
    name = "fs"
    service_delegation {
      name      = "Microsoft.DBforPostgreSQL/flexibleServers"
      actions   = ["Microsoft.Network/virtualNetworks/subnets/join/action"]
    }
  }
}

resource "azurerm_private_dns_zone" "flex" {
  name                       = "${local.name_prefix}.postgres.database.azure.com"
  resource_group_name       = azurerm_resource_group.network.name
}

resource "azurerm_private_dns_zone_virtual_network_link" "flex" {
  name                       = "flex-pdz-link"
  resource_group_name       = azurerm_resource_group.network.name
  private_dns_zone_name     = azurerm_private_dns_zone.flex.name
  virtual_network_id        = azurerm_virtual_network.this.id
}
```

7.3 Microsoft Entra ID authentication

Entra ID auth on Flexible Server uses tokens from `ossrdbms-aad.database.windows.net`. Terraform creates the AAD administrator and grants the resulting principal `CREATE ROLE` in the database; the application then logs in with its managed identity and an access token in lieu of a password:

```
resource "azurerm_postgresql_flexible_server_active_directory_administrator" "admin" {
  server_name      = azurerm_postgresql_flexible_server.this.name
  resource_group_name = azurerm_resource_group.db.name
  tenant_id       = data.azurerm_client_config.current.tenant_id
  object_id       = data.azuread_group.pg_admins.object_id
  principal_name   = data.azuread_group.pg_admins.display_name
  principal_type    = "Group"
}
```

7.4 Server parameters

Flexible Server parameters are individual resources, which makes them easy to read in a plan but verbose to declare. Wrap them in a `for_each`:

```
locals {
  pg_parameters = {
```

```

    "log_min_duration_statement" = "1000"
    "log_lock_waits"             = "on"
    "log_statement"              = "ddl"
    "pgaudit.log"                = "ddl,role,write"
    "shared_preload_libraries"   = "pg_stat_statements,pgaudit"
  }
}

resource "azurerm_postgresql_flexible_server_configuration" "params" {
  for_each = local.pg_parameters
  name     = each.key
  server_id = azurerm_postgresql_flexible_server.this.id
  value    = each.value
}

```

7.5 Read replicas

Flexible Server replicas are themselves `azurerm_postgresql_flexible_server` resources with `create_mode = "Replica"` and a `source_server_id`. Cross-region replicas are first-class:

```

resource "azurerm_postgresql_flexible_server" "replica_west_us" {
  provider           = azurerm.westus
  name               = "${local.name_prefix}-flex-westus"
  resource_group_name = azurerm_resource_group.dr.name
  location           = "westus2"

  create_mode        = "Replica"
  source_server_id    = azurerm_postgresql_flexible_server.this.id
  delegated_subnet_id = azurerm_subnet.db_westus.id
  private_dns_zone_id = azurerm_private_dns_zone.flex_westus.id

  sku_name      = "GP_Standard_D4ds_v5"
  storage_mb    = 524288

  lifecycle { ignore_changes = [administrator_password, zone, high_availability] }
}

```

Replica gotcha. A replica inherits server parameters from the source at creation, not continuously. If you change a parameter on the primary after the replica exists, you must apply it to the replica separately. Pair the parameter resources with `for_each` over a set of server IDs to keep both in sync.

Chapter 8 · State, Workspaces, and CI/CD for DBAs

Three operational disciplines separate a Terraform repository that works from one that scales: how state is stored, how environments are isolated, and how changes are reviewed and applied. This chapter is the DBA-flavoured guide to all three.

8.1 Remote backends

Local state on a laptop is a single-engineer anti-pattern. The minimum production backend is remote, encrypted at rest, and protected by IAM. The three reference setups, one per cloud, are:

```
# AWS: S3 + DynamoDB (or S3-native locking for Terraform 1.10+)
terraform {
  backend "s3" {
    bucket      = "acme-tf-state"
    key         = "db/prod/terraform.tfstate"
    region      = "eu-west-1"
    encrypt     = true
    kms_key_id  = "arn:aws:kms:eu-west-1:111122223333:key/abc"
    dynamodb_table = "tf-state-locks"
  }
}

# GCP: GCS with object versioning
terraform {
  backend "gcs" {
    bucket = "acme-tf-state"
    prefix = "db/prod"
  }
}

# Azure: storage account with state locking via blob lease
terraform {
  backend "azurerm" {
    resource_group_name = "rg-tfstate"
    storage_account_name = "acmetfstate"
    container_name       = "tfstate"
    key                  = "db/prod.tfstate"
  }
}
```

8.2 State surgery

Three commands and two blocks let you reshape state without recreating resources:

- `terraform import <address> <id>` — adopts an existing resource into state. Essential for ramping up a brownfield database.
- `terraform state mv <from> <to>` — renames a state address without touching the real resource.

- `terraform state rm <address>` — forgets a resource. The real thing remains; Terraform stops managing it.
- `moved { from = ... to = ... }` block (Terraform 1.1+) — declarative rename, committed to the repository and applied automatically on next plan.
- `removed { from = ... }` block (Terraform 1.7+) — declarative `state rm`, optionally with a destroy provisioner.

```

moved {
  from = aws_db_instance.legacy
  to   = aws_db_instance.primary
}

# 1.7+: forget the old read replica without destroying it
removed {
  from = aws_db_instance.read_replica["us-east-1c"]
  lifecycle { destroy = false }
}

```

8.3 Drift detection

Drift is the silent killer. A scheduled CI job runs `terraform plan -detailed-exitcode` against production every six hours and pages on a non-zero exit code. The plan output is posted to a Slack channel for review. For databases, drift commonly comes from:

- An engineer using the cloud console for an emergency change.
- Cloud-side automatic minor version upgrades (set `auto_minor_version_upgrade = false` and accept that you must drive patching).
- Cloud-side zone reassignments after a regional event.
- Parameter groups left in `pending-reboot` state for months.

8.4 CI/CD pipelines

A correct pipeline runs `plan` on pull requests and `apply` on merge to main, with a manual gate before production. A representative GitHub Actions job:

```

name: terraform
on:
  pull_request: { paths: ['envs/**', 'modules/**'] }
  push: { branches: [main] }

permissions:
  id-token: write    # for OIDC to AWS
  contents: read

jobs:
  plan:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        env: [dev, staging, prod]
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3

```

```

with: { terraform_version: 1.9.5 }
- uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: arn:aws:iam::111122223333:role/tf-ci-${{ matrix.env }}
    aws-region: eu-west-1
- run: terraform -chdir=envs/${{ matrix.env }} init
- run: terraform -chdir=envs/${{ matrix.env }} fmt -check
- run: terraform -chdir=envs/${{ matrix.env }} validate
- run: |
  terraform -chdir=envs/${{ matrix.env }} plan \
    -out=tfplan -detailed-exitcode
  continue-on-error: true
  id: plan
- run: terraform -chdir=envs/${{ matrix.env }} show -no-color tfplan &gt; plan.txt
- uses: actions/upload-artifact@v4
  with:
    name: plan-${{ matrix.env }}
    path: envs/${{ matrix.env }}/tfplan

apply-prod:
  needs: plan
  if: github.ref == 'refs/heads/main'
  environment: production # GitHub Environment with required reviewers
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
      with: { name: plan-prod, path: envs/prod }
    - run: terraform -chdir=envs/prod apply tfplan

```

OIDC, not long-lived keys. CI should assume an AWS/GCP/Azure role via OIDC, never use a static credential. Static credentials in CI logs are the single most common breach vector for Terraform-managed environments.

8.5 Higher-level orchestrators

When the repository grows past a handful of environments, the raw GitHub Actions pattern strains. Three established alternatives:

Tool	Model	When to choose
Atlantis	Self-hosted Git webhook server; plan in PR comments	Strongly for teams that want full control and a small footprint.
Terraform Cloud / HCP	SaaS; PR integration, Sentinel policy-as-code, drift detection	Easy to get up and running with zero infra to operate and have HashiCorp backing.
Spacelift	SaaS; pluggable policy (OPA), stacks, drift detection	Teams who want OPA over Sentinel or who have many stacks.

Chapter 9 · Security, Compliance, and Day-2 Operations

This final chapter is the operational checklist the DBA reaches for when the architecture is settled and the question becomes *is this production-ready*. Each section is a discrete production discipline; all of them should be present in any database stack worth running.

9.1 Secret management

Three rules govern secrets in a Terraform-managed database:

- **Generate, never write.** Use `random_password` with a `keepers` map for the rotation handle.
- **Write once, read everywhere.** Store the credential in the platform-native secret manager (AWS Secrets Manager, Google Secret Manager, Azure Key Vault); applications read the secret at boot.
- **Rotate on a clock.** AWS Secrets Manager has built-in rotation; for GCP and Azure, schedule a rotation Lambda/Function or use the rotation handle pattern from Chapter 3.

9.2 Network surface minimisation

- No public IP. Always. The cost of a bastion is lower than the cost of one breach.
- Security groups (or VPC firewall rules) that allow *only* the application security group on port 5432. No `0.0.0.0/0`, ever.
- VPC flow logs enabled and shipped to a SIEM. Most database breaches are detected in flow logs first, in the database logs second.
- TLS required at the engine level (`rds.force_ssl = 1`, Cloud SQL `ssl_mode = "ENCRYPTED_ONLY"`, Azure `require_secure_transport = on`).

9.3 Encryption at rest

Every managed PostgreSQL service supports encryption at rest with customer-managed keys (CMK). Terraform should provision the key, set the rotation policy, and reference it from the database resource:

```
resource "aws_kms_key" "rds" {
  description      = "CMK for ${local.name_prefix} RDS"
  enable_key_rotation = true
  deletion_window_in_days = 30
  policy           = data.aws_iam_policy_document.rds_kms.json
}

resource "aws_kms_alias" "rds" {
  name         = "alias/${local.name_prefix}-rds"
  target_key_id = aws_kms_key.rds.id
}
```

9.4 Audit logging

`pgaudit` is the industry standard. Enable it via the parameter group (RDS), database flag (Cloud SQL with `cloudsql.enable_pgaudit`) or server parameter (Azure Flexible Server). The DBA-relevant configuration is `pgaudit.log = "ddl, role, write"` — capturing the events auditors actually ask about. Capturing read as well is noisy and expensive; do it only for explicitly regulated tables via `pgaudit.role`.

9.5 Backup verification

A backup that has never been restored is not a backup. Schedule a monthly restore-test job that:

- Creates a temporary instance from the most recent automated snapshot.
- Runs a known-good checksum query (`SELECT count(*)` on a few tables, plus a `pg_dump | sha256sum` of a stable schema).
- Reports the checksum to a tracking system and destroys the temporary instance.

Terraform participates by providing the temporary instance as a `terraform apply` against a scratch workspace, parameterised by the snapshot ID. The job runs in CI on a cron schedule and pages on checksum mismatch.

9.6 Compliance annotations

Most regulated environments require that infrastructure resources be *labelled* with their compliance classification. Encode this in tags so the compliance team can inventory everything from the cloud's tag-search:

```
locals {
  compliance_tags = {
    DataClassification = "pii"
    Compliance        = "pci-dss-3.2.1,hipaa,soc2"
    Owner              = "team-payments"
    BackupTier         = "tier1"
    RTO                = "1h"
    RPO                = "5m"
  }
}

provider "aws" {
  default_tags { tags = local.compliance_tags }
}
```

9.7 Cost controls

- Tag every resource with `CostCenter` and `Environment`; pipe the cloud cost reports into FinOps tooling.
- Use Aurora Serverless v2 or AlloyDB's minimum sizing for non-prod.
- Right-size storage IOPS; gp3's baseline 3000 IOPS is enough for most workloads, paying for io2 only when you have measured the need.

- Schedule non-prod environments to stop overnight — this is a terraform plan on `aws_db_instance.dev.instance_class = "stopped"` equivalent, driven by an EventBridge rule.

9.8 Day-2 runbooks the DBA should own

Terraform’s ergonomics for the following are deliberate — the actions exist, they should be in your repository, and they should be rehearsed:

Runbook	Terraform actions involved
Promote a read replica	Edit Terraform to remove the source/replica relationship; <code>terraform state rm aws_db_instance.replica</code>
Scale up the writer	Edit <code>instance_class</code> ; <code>apply</code> in a maint window
Recover from a snapshot	Apply a sibling module that creates a new instance from <code>snapshot_id</code>
Rotate the master credential	Bump <code>var.rotation_id</code> ; <code>apply</code> ; redeploy IAM role
Major version upgrade	Enable <code>blue_green_update</code> ; initiate cutover via cloud API; <code>terraform apply</code>

The discipline is the deliverable. A repository that contains the runbooks, the modules, the CI/CD pipeline, the drift detection, and the compliance annotations is a deliverable in its own right. It outlives the individual DBA. It is the artefact this book has been building toward.

Part II · Capstone Projects

The five projects in this section bring together the techniques from the preceding chapters. Each is sized for a single working day: roughly two hours of reading, two hours of typing, and four hours of discovering what your cloud provider quietly changed since the docs were last updated. The projects are independent — you can do them in any order — but they reinforce each other if read in sequence.

#	Project	Target service
1	Encrypted RDS with Roles and Backups	AWS RDS for PostgreSQL
2	HA Self-Hosted Pair with Patroni	EC2 + Patroni + etcd
3	Point-in-Time Recovery Pipeline	GCP Cloud SQL
4	RBAC and Secret Rotation	Azure Flexible Server
5	GitOps Schema Migrations	Atlas + GitHub Actions

Project 1 · Encrypted RDS with Roles and Backups

Objective

Provision a production-grade RDS for PostgreSQL instance: encrypted with a customer-managed KMS key, deployed in a private subnet group, fronted by RDS Proxy, audited with pgaudit, and populated with three application databases each owned by a least-privilege role whose credentials live in Secrets Manager. The whole stack should be destructible and rebuildable with a single command.

Prerequisites

- An AWS account with an existing VPC and at least two private subnets tagged `Tier = database`.
- Terraform 1.9+ and the AWS CLI configured for the target account.
- Permission to create KMS keys, IAM roles, RDS resources and Secrets Manager secrets.

Architecture

Three resource layers. The substrate layer creates the KMS key, the security group, the subnet group, and the parameter group. The instance layer creates the RDS instance and the RDS Proxy. The in-database layer creates the three application databases and roles via the PostgreSQL provider.

Step 1 — Repository layout

```
project-1/
  versions.tf      # required_providers + backend
  main.tf          # substrate
  rds.tf           # instance + proxy
  in_db.tf         # databases and roles (cyrilgdn/postgresql)
  variables.tf
  outputs.tf
  terraform.tfvars # values for this run
```

Step 2 — Substrate

```
# main.tf
data "aws_vpc" "this" { id = var.vpc_id }
data "aws_subnets" "db" {
  filter { name = "vpc-id" values = [var.vpc_id] }
  filter { name = "tag:Tier" values = ["database"] }
}

resource "aws_kms_key" "rds" {
  description          = "CMK for project-1 RDS"
  enable_key_rotation  = true
  deletion_window_in_days = 30
}

resource "aws_kms_alias" "rds" {
  name          = "alias/project-1-rds"
  target_key_id = aws_kms_key.rds.id
}

resource "aws_security_group" "db" {
  name          = "project-1-db"
  description    = "Allow PostgreSQL from app SG"
  vpc_id        = var.vpc_id

  ingress {
    from_port      = 5432
    to_port        = 5432
    protocol       = "tcp"
    security_groups = var.app_security_group_ids
  }
  egress {
    from_port = 0 to_port = 0 protocol = "-1" cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_db_subnet_group" "this" {
  name          = "project-1-subnets"
  subnet_ids    = data.aws_subnets.db.ids
}

resource "aws_db_parameter_group" "pg16" {
  name     = "project-1-pg16"
  family   = "postgres16"

  parameter { name = "shared_preload_libraries" value = "pg_stat_statements,pgaudit" apply_method =
  parameter { name = "rds.force_ssl" value = "1" apply_method =
  parameter { name = "log_min_duration_statement" value = "500" }
  parameter { name = "log_lock_waits" value = "1" }
  parameter { name = "pgaudit.log" value = "ddl, role, write" }
}
```

Step 3 — The RDS instance

```
# rds.tf
resource "aws_db_instance" "this" {
  identifier          = "project-1"
  engine              = "postgres"
  engine_version      = "16.3"
  instance_class      = "db.r6g.large"
  allocated_storage   = 100
  max_allocated_storage = 500
  storage_type        = "gp3"
  storage_encrypted    = true
  kms_key_id          = aws_kms_key.rds.arn

  username            = "rds_admin"
  manage_master_user_password = true
  master_user_secret_kms_key_id = aws_kms_key.rds.arn

  db_subnet_group_name = aws_db_subnet_group.this.name
  vpc_security_group_ids = [aws_security_group.db.id]
  parameter_group_name = aws_db_parameter_group.pg16.name

  multi_az          = true
  publicly_accessible = false
  backup_retention_period = 35
  deletion_protection = true
  copy_tags_to_snapshot = true
  performance_insights_enabled = true
  performance_insights_retention_period = 731
  enabled_cloudwatch_logs_exports = ["postgresql"]
  iam_database_authentication_enabled = true
  auto_minor_version_upgrade = false
  apply_immediately = false

  lifecycle { prevent_destroy = true }
}

resource "aws_db_proxy" "this" {
  name          = "project-1"
  engine_family = "POSTGRESQL"
  role_arn      = aws_iam_role.proxy.arn
  vpc_subnet_ids = data.aws_subnets.db.ids
  vpc_security_group_ids = [aws_security_group.db.id]
  require_tls     = true
  idle_client_timeout = 1800

  auth {
    auth_scheme = "SECRETS"
    iam_auth    = "DISABLED"
    secret_arn  = aws_db_instance.this.master_user_secret[0].secret_arn
  }
}

resource "aws_db_proxy_default_target_group" "this" {
  db_proxy_name = aws_db_proxy.this.name
  connection_pool_config {
    max_connections_percent = 75
    max_idle_connections_percent = 25
  }
}
```

```

resource "aws_db_proxy_target" "this" {
  db_proxy_name      = aws_db_proxy.this.name
  target_group_name  = aws_db_proxy_default_target_group.this.name
  db_instance_identifier = aws_db_instance.this.identifier
}

```

Step 4 — In-database resources

Fetch the admin password from the AWS-managed secret and use it to configure the PostgreSQL provider. Then declare three databases, each with its own role and least-privilege grants:

```

# in_db.tf
data "aws_secretsmanager_secret_version" "admin" {
  secret_id = aws_db_instance.this.master_user_secret[0].secret_arn
}

locals {
  admin = jsondecode(data.aws_secretsmanager_secret_version.admin.secret_string)
  databases = {
    orders = { owner_role = "orders_app" }
    catalog = { owner_role = "catalog_app" }
    audit = { owner_role = "audit_writer" }
  }
}

provider "postgresql" {
  host      = aws_db_instance.this.address
  port      = aws_db_instance.this.port
  database  = "postgres"
  username  = local.admin.username
  password  = local.admin.password
  superuser = false
  sslmode   = "require"
}

resource "random_password" "app" {
  for_each = local.databases
  length   = 32
  special  = true
}

resource "aws_secretsmanager_secret" "app" {
  for_each = local.databases
  name     = "project-1/${each.key}"
  kms_key_id = aws_kms_key.rds.id
}

resource "aws_secretsmanager_secret_version" "app" {
  for_each = local.databases
  secret_id = aws_secretsmanager_secret.app[each.key].id
  secret_string = jsonencode({
    username = each.value.owner_role
    password = random_password.app[each.key].result
    host     = aws_db_proxy.this.endpoint
    port     = 5432
    dbname   = each.key
  })
}

resource "postgresql_role" "app" {
  for_each = local.databases
}

```

```

    name          = each.value.owner_role
    login          = true
    password       = random_password.app[each.key].result
    encrypted_password = true
    connection_limit = 50
    lifecycle { ignore_changes = [password] }
  }

  resource "postgresql_database" "app" {
    for_each      = local.databases
    name          = each.key
    owner         = postgresql_role.app[each.key].name
    encoding      = "UTF8"
    lc_collate    = "en_US.utf8"
    template      = "template0"
    connection_limit = 100
    allow_connections = true
    lifecycle { prevent_destroy = true }
  }

  resource "postgresql_extension" "pg_stat_statements" {
    for_each = local.databases
    name     = "pg_stat_statements"
    database = postgresql_database.app[each.key].name
  }

```

Step 5 — Apply

Because of the chicken-and-egg bootstrap, apply in two phases:

```

terraform init
terraform apply -target=aws_db_instance.this -target=aws_db_proxy.this
terraform apply

```

Step 6 — Validation

- Connect through the proxy with one of the application credentials and confirm `SELECT current_database( );` returns the expected name.
- Run `\du+` in psql and confirm no role has the `SUPERUSER` attribute.
- Trigger a manual snapshot via the AWS console; verify the snapshot is encrypted with your CMK (`aws rds describe-db-snapshots`).
- Confirm pgaudit is logging by running a DDL statement and tailing CloudWatch Logs.

Extensions

- Wire IAM authentication for a CI service account using the pattern in §5.4.
- Add a cross-region read replica and verify the replica inherits the parameter group.
- Add a Lambda-driven rotation function to Secrets Manager.

Project 2 · HA Self-Hosted Pair with Patroni

Objective

Build a two-node PostgreSQL cluster (primary + synchronous standby) on EC2 with Patroni for leader election against a three-node etcd. Front the cluster with a Network Load Balancer whose health check follows the Patroni leader. Verify automatic failover by terminating the primary's instance and watching the standby take over within 30 seconds.

Prerequisites

- An AWS account, a VPC with private subnets in at least three AZs.
- Terraform 1.9+, AWS CLI, `psql`.
- Quota for `r6i.xlarge` instances in three AZs.

Architecture

etcd cluster: three `t3.small` instances, one per AZ. **PostgreSQL cluster:** two `r6i.xlarge` instances, primary in AZ-a, standby in AZ-b, both joined to the etcd quorum. **NLB:** one internal NLB with a target group whose health check polls Patroni's `/primary` endpoint.

Step 1 — Module skeleton

```
project-2/  
  main.tf  
  etcd.tf  
  postgres.tf  
  nlb.tf  
  cloud-init/  
    etcd.yaml.tftpl  
    postgres.yaml.tftpl  
  variables.tf  
  outputs.tf
```

Step 2 — etcd cluster

```

locals {
  etcd_nodes = {
    e1 = { az = "eu-west-1a", subnet = var.subnet_a }
    e2 = { az = "eu-west-1b", subnet = var.subnet_b }
    e3 = { az = "eu-west-1c", subnet = var.subnet_c }
  }
}

resource "aws_instance" "etcd" {
  for_each      = local.etcd_nodes
  ami           = data.aws_ami.ubuntu_22.id
  instance_type = "t3.small"
  subnet_id     = each.value.subnet
  vpc_security_group_ids = [aws_security_group.etcd.id]
  iam_instance_profile = aws_iam_instance_profile.etcd.name

  user_data = templatefile("${path.module}/cloud-init/etcd.yaml.tftpl", {
    node_name = each.key
    cluster   = join(",", [for k, v in local.etcd_nodes :
                          "${k}=https://${v}.etcd.local:2380"])
  })
  tags = { Name = "etcd-${each.key}", Role = "etcd" }
}

resource "aws_route53_record" "etcd" {
  for_each = aws_instance.etcd
  zone_id  = aws_route53_zone.internal.id
  name     = "${each.key}.etcd.local"
  type     = "A"
  ttl      = 30
  records  = [each.value.private_ip]
}

```

Step 3 — PostgreSQL nodes

```

locals {
  pg_nodes = {
    pg1 = { az = "eu-west-1a", subnet = var.subnet_a }
    pg2 = { az = "eu-west-1b", subnet = var.subnet_b }
  }
}

resource "aws_instance" "pg" {
  for_each      = local.pg_nodes
  ami           = data.aws_ami.ubuntu_22.id
  instance_type = "r6i.xlarge"
  subnet_id     = each.value.subnet
  vpc_security_group_ids = [aws_security_group.pg.id]
  iam_instance_profile = aws_iam_instance_profile.pg.name
  ebs_optimized = true

  root_block_device {
    volume_type = "gp3" volume_size = 50 encrypted = true
  }

  metadata_options { http_tokens = "required" }

  user_data = templatefile("${path.module}/cloud-init/postgres.yaml.tftpl", {
    node_name      = each.key
    cluster_name   = "pg-prod"
    etcd_endpoints = join(",", [for k, _ in local.etcd_nodes :
                                "https://${k}.etcd.local:2379"])
    backup_bucket  = aws_s3_bucket.backups.id
  })
  lifecycle { ignore_changes = [user_data, ami] }
}

resource "aws_ebs_volume" "pg_data" {
  for_each      = aws_instance.pg
  availability_zone = each.value.availability_zone
  size          = 500
  type          = "gp3"
  iops          = 12000
  throughput    = 500
  encrypted     = true
}

resource "aws_volume_attachment" "pg_data" {
  for_each      = aws_ebs_volume.pg_data
  device_name   = "/dev/xvdf"
  volume_id     = each.value.id
  instance_id   = aws_instance.pg[each.key].id
}

```

Step 4 — NLB and leader-following target group

```
resource "aws_lb" "pg" {
  name           = "pg-prod"
  internal       = true
  load_balancer_type = "network"
  subnets       = [var.subnet_a, var.subnet_b]
}

resource "aws_lb_target_group" "pg_primary" {
  name        = "pg-prod-primary"
  port        = 5432
  protocol    = "TCP"
  vpc_id      = var.vpc_id
  target_type = "instance"

  health_check {
    protocol          = "HTTP"
    port              = "8008"
    path              = "/primary"
    interval          = 5
    healthy_threshold = 2
    unhealthy_threshold = 2
  }
}

resource "aws_lb_target_group_attachment" "pg_primary" {
  for_each      = aws_instance.pg
  target_group_arn = aws_lb_target_group.pg_primary.arn
  target_id     = each.value.id
}

resource "aws_lb_listener" "pg" {
  load_balancer_arn = aws_lb.pg.arn
  port              = 5432
  protocol          = "TCP"
  default_action {
    type = "forward"
    target_group_arn = aws_lb_target_group.pg_primary.arn
  }
}
```

Step 5 — Validation

- Connect with `psql -h <nlb-dns> -U postgres`. Verify `SELECT pg_is_in_recovery();` returns `f`.
- Query the Patroni REST API on each node: `curl <node>:8008/cluster | jq`.
- Terminate the current primary instance via the AWS console. Watch the NLB target health flip within ~10 seconds. Reconnect with `psql` and confirm the new primary.

Cleanup

```
terraform state rm 'aws_ebs_volume.pg_data["pg1"]' # preserve data
terraform state rm 'aws_ebs_volume.pg_data["pg2"]'
terraform destroy
```

Project 3 · Point-in-Time Recovery Pipeline (Cloud SQL)

Objective

Build a Cloud SQL for PostgreSQL instance with point-in-time recovery enabled, and a scheduled workflow that exercises PITR end-to-end: every night, a Cloud Workflow restores the production database to a scratch instance at $T - 1 \text{ hour}$, runs a checksum query against a canary table, exports the result to BigQuery, and destroys the scratch instance. The Terraform stack manages everything except the workflow logic, which lives as a YAML in the same repository.

Prerequisites

- GCP project with Cloud SQL Admin, Cloud Workflows, BigQuery and Secret Manager APIs enabled.
- A VPC configured with PSA or PSC; Terraform 1.9+ and gcloud CLI.

Architecture

Production: one `google_sql_database_instance` with `point_in_time_recovery_enabled = true` and 7 days of transaction-log retention. **Restore pipeline:** a Cloud Workflow, triggered nightly by Cloud Scheduler, that issues the `cloneInstance` API call with a `pointInTime` argument. **Verification:** a Cloud Function that connects to the cloned instance, runs the canary query, and writes the result to BigQuery.

Step 1 — Production instance

```
resource "google_sql_database_instance" "prod" {
  name                = "project-3-prod"
  database_version    = "POSTGRES_16"
  region              = var.region
  deletion_protection = true

  settings {
    tier                = "db-custom-4-16384"
    availability_type   = "REGIONAL"
    disk_type           = "PD_SSD"
    disk_size           = 200
    disk_autoresize     = true

    backup_configuration {
      enabled                = true
      point_in_time_recovery_enabled = true
      start_time              = "02:00"
      transaction_log_retention_days = 7
      backup_retention_settings {
        retained_backups = 30
        retention_unit   = "COUNT"
      }
    }
  }

  ip_configuration {
    ipv4_enabled = false
    private_network = google_compute_network.vpc.id
    ssl_mode       = "ENCRYPTED_ONLY"
  }

  database_flags { name = "cloudsql.enable_pgaudit" value = "on" }
  database_flags { name = "pgaudit.log" value = "ddl,role,write" }

  insights_config {
    query_insights_enabled = true
    query_string_length    = 4500
  }
}

resource "google_sql_database" "app" {
  name       = "appdb"
  instance   = google_sql_database_instance.prod.name
}

resource "google_sql_user" "app" {
  name       = "app"
  instance   = google_sql_database_instance.prod.name
  password   = random_password.app.result
}
```

Step 2 — Restore workflow

```
resource "google_workflows_workflow" "pitrr_drill" {
  name          = "pitrr-drill"
  region        = var.region
  service_account = google_service_account.workflow.id
  source_contents = file("${path.module}/workflows/pitrr-drill.yaml")
}

resource "google_cloud_scheduler_job" "pitrr_nightly" {
  name          = "pitrr-drill-nightly"
  region        = var.region
  schedule      = "0 3 * * *"
  time_zone     = "UTC"
  http_target {
    http_method = "POST"
    uri         = "https://workflowexecutions.googleapis.com/v1/projects/${var.project_id}/locations/${var.region}/workflows/${google_workflows_workflow.pitrr_drill.name}/execute"
    oauth_token { service_account_email = google_service_account.scheduler.email }
  }
}
```

The workflow YAML, abbreviated:

```
main:
  steps:
    - calc_target_time:
      assign:
        - target_time: ${time.format(sys.now() - 3600)} # one hour ago
        - clone_name: ${"project-3-pitr-" + string(int(sys.now()))}
    - clone:
      call: googleapis.sqladmin.v1.instances.clone
      args:
        project: ${var.project_id}
        instance: project-3-prod
        body:
          cloneContext:
            destinationInstanceName: ${clone_name}
            pointInTime: ${target_time}
    - wait_ready:
      call: googleapis.sqladmin.v1.operations.wait
      args:
        operation: ${clone.name}
        timeoutSec: 1800
    - verify:
      call: http.post
      args:
        url: ${"https://" + var.region + "-" + var.project_id + ".cloudfunctions.net/pitr-verify"}
        auth: { type: OIDC }
        body: { instance: ${clone_name} }
    - destroy:
      call: googleapis.sqladmin.v1.instances.delete
      args:
        project: ${var.project_id}
        instance: ${clone_name}
```

Step 3 — The verification function

A small Cloud Function in Python connects to the clone, runs a canary query (`SELECT count(*) FROM orders` on a stable hour-old row count), and inserts the result into BigQuery. The Terraform side:

```

resource "google_cloudfunctions2_function" "pitr_verify" {
  name     = "pitr-verify"
  location = var.region

  build_config {
    runtime      = "python311"
    entry_point  = "verify"
    source { storage_source {
      bucket = google_storage_bucket.functions.name
      object = google_storage_bucket_object.verify_source.name
    } }
  }

  service_config {
    available_memory = "256M"
    timeout_seconds  = 300
    service_account_email = google_service_account.verify.email
    vpc_connector        = google_vpc_access_connector.functions.id
    environment_variables = {
      BQ_TABLE = "${var.project_id}.audit.pitr_drill"
    }
  }
}

```

Step 4 — Validation

- Run the workflow manually: `gcloud workflows execute pitr-drill`. Observe a fresh clone appear in Cloud SQL and disappear ~15 minutes later.
- Query `audit.pitr_drill` in BigQuery to confirm the row was inserted with status OK.
- Manually corrupt the canary table on production, then re-run the workflow with a pre-corruption `pointInTime`: the verification should still report OK because PITR restored to a clean state.

Cost note. A Cloud SQL clone costs the equivalent of a fresh instance for the duration it exists. The 15-minute nightly drill is inexpensive (a few dollars a month at the suggested tier); leaving the clone alive accidentally is not. Make sure the workflow's destroy step is wrapped in a try/retry block and that you have an alert on Cloud SQL instances older than 2 hours with names matching `project-3-pitr-*`.

Project 4 · RBAC and Secret Rotation on Azure Flexible Server

Objective

Deploy an Azure Database for PostgreSQL Flexible Server with two application databases. Establish a four-role permission model (*admin*, *migrator*, *read-write*, *read-only*) implemented in Terraform with no plaintext passwords anywhere — all credentials are generated by `random_password`, stored in Azure Key Vault, and rotated quarterly by an Azure Function that this stack deploys. Application identities authenticate via Microsoft Entra ID.

Architecture

Network: a delegated subnet in a hub VNet, with a private DNS zone for the server. **Server:** one zone-redundant Flexible Server with Entra ID auth enabled. **Vault:** one Key Vault with per-role secrets. **Rotation:** one Function App, triggered by a timer, that bumps the `rotation_id` variable in a remote-state file and runs a thin `terraform apply`.

Step 1 — Network and server

```
resource "azurerm_postgresql_flexible_server" "this" {
  name                = "project-4-pg"
  resource_group_name = azurerm_resource_group.this.name
  location            = azurerm_resource_group.this.location
  version             = "16"

  delegated_subnet_id = azurerm_subnet.pg.id
  private_dns_zone_id = azurerm_private_dns_zone.pg.id

  administrator_login      = "pg_admin"
  administrator_password = random_password.pg_admin.result

  sku_name      = "GP_Standard_D4ds_v5"
  storage_mb    = 524288
  zone          = "1"

  high_availability {
    mode                = "ZoneRedundant"
    standby_availability_zone = "2"
  }

  backup_retention_days      = 35
  geo_redundant_backup_enabled = true

  authentication {
    active_directory_auth_enabled = true
    password_auth_enabled        = true
    tenant_id                    = data.azurerm_client_config.current.tenant_id
  }

  lifecycle {
    prevent_destroy = true
    ignore_changes = [administrator_password, zone]
  }
}
```

Step 2 — The four-role model

```

locals {
  databases = ["orders", "catalog"]
  roles = {
    admin = {
      attributes = { createrole = true, createdb = true }
      privileges = ["ALL"]
    }
    migrator = {
      attributes = {}
      privileges = ["CREATE", "USAGE"] # schema-level
    }
    rw = {
      attributes = {}
      privileges = ["SELECT", "INSERT", "UPDATE", "DELETE", "TRUNCATE"]
    }
    ro = {
      attributes = {}
      privileges = ["SELECT"]
    }
  }
}

resource "random_password" "role" {
  for_each = local.roles
  length   = 32
  special  = true
  keepers  = { rotation_id = var.rotation_id }
}

resource "postgresql_role" "this" {
  for_each      = local.roles
  name          = each.key
  login         = true
  password      = random_password.role[each.key].result
  encrypted_password = true
  create_role   = lookup(each.value.attributes, "createrole", false)
  create_database = lookup(each.value.attributes, "createdb", false)
  lifecycle { ignore_changes = [password] }
}

resource "postgresql_database" "this" {
  for_each = toset(local.databases)
  name     = each.value
  owner    = postgresql_role.this["admin"].name
  template = "template0"
  lifecycle { prevent_destroy = true }
}

resource "postgresql_grant" "rw" {
  for_each      = toset(local.databases)
  database      = postgresql_database.this[each.value].name
  role          = postgresql_role.this["rw"].name
  schema        = "public"
  object_type   = "table"
  privileges    = local.roles.rw.privileges
}

resource "postgresql_default_privileges" "rw_future" {
  for_each = toset(local.databases)
}

```

```

    database      = postgresql_database.this[each.value].name
    role          = postgresql_role.this["rw"].name
    owner         = postgresql_role.this["migrator"].name
    schema        = "public"
    object_type   = "table"
    privileges    = local.roles.rw.privileges
  }

  # repeat the grant / default_privileges pair for ro, migrator

```

Step 3 — Secrets in Key Vault

```

resource "azurerm_key_vault" "this" {
  name                        = "kv-project-4"
  resource_group_name        = azurerm_resource_group.this.name
  location                   = azurerm_resource_group.this.location
  tenant_id                  = data.azurerm_client_config.current.tenant_id
  sku_name                   = "standard"
  purge_protection_enabled   = true
  soft_delete_retention_days = 30

  enable_rbac_authorization = true
}

resource "azurerm_key_vault_secret" "role" {
  for_each = local.roles
  name     = "pg-${each.key}"
  value    = random_password.role[each.key].result
  key_vault_id = azurerm_key_vault.this.id

  content_type = "application/json"
  tags = {
    Role      = each.key
    Rotation  = var.rotation_id
  }
}

```

Step 4 — The rotation function

The Function App is deployed as a `azurerm_linux_function_app` with an HTTP trigger. The function's body opens the repository's remote state, bumps `rotation_id` in a `terraform.tfvars.json` file committed to a config repo, and uses the GitHub API to open a pull request that auto-merges if CI passes. The credential rotation therefore follows the same review path as every other change.

Validation

- Connect with each role's credential and verify `\dp` output matches the matrix.
- Force a rotation by setting `rotation_id = 2`; verify Key Vault secret versions increment and the role's old password fails.
- Confirm Entra ID auth: log in as a privileged AAD user using `az account get-access-token --resource https://ossrdbms-aad.database.windows.net`.

Project 5 · GitOps Schema Migrations with Atlas

Objective

Build a GitOps pipeline that promotes PostgreSQL schema changes through dev → staging → production using Atlas as the migration engine. Terraform manages the database infrastructure; Atlas manages everything inside the public schema. Migration code and infrastructure code live in the same repository and share the same review process.

Why not Terraform for schema?

Schema migrations have ordering, transactional, and rollback semantics that Terraform's plan/apply model does not respect. Atlas (or Flyway, or Sqitch) is a state-machine migrator with a first-class understanding of *versioned* changes. The right division of labour is: Terraform builds the substrate and the in-database roles/databases/grants; Atlas owns DDL inside the schema. The integration point is the connection string Terraform writes to a secret that Atlas reads at migration time.

Repository layout

```
project-5/
  infra/                                # Terraform
    envs/dev/main.tf
    envs/staging/main.tf
    envs/prod/main.tf
    modules/db/...
  schema/                               # Atlas
    schema.hcl                          # desired schema
    migrations/                         # generated, versioned
      20260420093015_initial.sql
      20260502141021_add_orders_idx.sql
    atlas.hcl                           # config
  .github/workflows/
    terraform.yaml
    atlas.yaml
```

Step 1 — The Atlas config

```
# schema/atlas.hcl
variable "db_url" { type = string }

env "dev" {
  src = "file://schema.hcl"
  url = var.db_url
  dev = "docker://postgres/16/dev"      # ephemeral linter DB
  migration { dir = "file://migrations" }
  format {
    migrate { diff = "{{ sql . \" \" }}" }
  }
}

env "staging" {
  src = "file://schema.hcl"
  url = var.db_url
  dev = "docker://postgres/16/dev"
  migration { dir = "file://migrations" }
}

env "prod" {
  src = "file://schema.hcl"
  url = var.db_url
  dev = "docker://postgres/16/dev"
  migration { dir = "file://migrations" }
}
```

Step 2 — The Atlas pipeline

```
# .github/workflows/atlas.yaml
name: schema
on:
  pull_request: { paths: ['schema/**'] }
  push:          { branches: [main], paths: ['schema/**'] }

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: ariga/setup-atlas@v0
        with: { cloud-token: ${ secrets.ATLAS_CLOUD_TOKEN } }
      - run: atlas migrate lint --dir file://schema/migrations \
          --dev-url docker://postgres/16/dev \
          --latest 1

  apply-staging:
    needs: lint
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v4
      - uses: ariga/setup-atlas@v0
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.STAGING_ROLE }
          aws-region: eu-west-1
      - run: |
          DB_URL=$(aws secretsmanager get-secret-value \
            --secret-id project-5/staging/migrator \
            --query SecretString --output text | jq -r .url)
          atlas migrate apply --env staging --url "$DB_URL"

  apply-prod:
    needs: apply-staging
    runs-on: ubuntu-latest
    environment: production # required-reviewer gate
    steps:
      - uses: actions/checkout@v4
      - uses: ariga/setup-atlas@v0
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ secrets.PROD_ROLE }
          aws-region: eu-west-1
      - run: |
          DB_URL=$(aws secretsmanager get-secret-value \
            --secret-id project-5/prod/migrator \
            --query SecretString --output text | jq -r .url)
          atlas migrate apply --env prod --url "$DB_URL"
```

Step 3 — The migrator role

Terraform creates the migrator role with exactly the privileges Atlas needs (CREATE in the schema, ownership of tables it has created):

```
resource "postgresql_role" "migrator" {
```

```

    name          = "migrator"
    login          = true
    password       = random_password.migrator.result
    encrypted_password = true
    create_role    = false
    create_database = false
    lifecycle { ignore_changes = [password] }
}

resource "postgresql_grant" "migrator_schema" {
  database = postgresql_database.app.name
  role     = postgresql_role.migrator.name
  schema   = "public"
  object_type = "schema"
  privileges = ["CREATE", "USAGE"]
}

resource "aws_secretsmanager_secret" "migrator" {
  name = "project-5/${var.environment}/migrator"
}

resource "aws_secretsmanager_secret_version" "migrator" {
  secret_id = aws_secretsmanager_secret.migrator.id
  secret_string = jsonencode({
    url = "postgres://migrator:${urlencode(random_password.migrator.result)}@${aws_db_instance.this.address}
  })
}

```

Step 4 — The schema file

Atlas's `schema.hcl` describes the desired state. Engineers edit this file; Atlas diffs it against the database and generates a versioned migration:

```

# schema/schema.hcl
schema "public" {}

table "orders" {
  schema = schema.public
  column "id" { type = uuid    null = false default = sql("gen_random_uuid()") }
  column "customer_id" { type = bigint null = false }
  column "total_cents" { type = bigint null = false }
  column "created_at" { type = timestamptz null = false default = sql("now()") }
  primary_key { columns = [column.id] }
  index "orders_customer_idx" { columns = [column.customer_id] }
}

```

Validation

- Edit `schema.hcl` to add a column; run `atlas migrate diff add_status_column --env dev`; verify a timestamped migration appears.
- Open a PR; verify `atlas migrate lint` runs and posts a comment.
- Merge to `main`; verify staging applies automatically; verify production waits for the required reviewer.
- Force a destructive change (a column rename); verify Atlas's lint reports the destructive operation and the PR is blocked until explicitly approved.

The handshake between Terraform and Atlas. Terraform writes the migrator's connection string to a secret; Atlas reads it. Neither tool reaches into the other's domain. This separation is the central abstraction of the project — one Terraform run rebuilds the instance, one Atlas run rebuilds the schema, and neither has to know the implementation details of the other.

Appendix A · Glossary

Term	Definition
Backend	Where Terraform stores its state file. The local backend (default) writes to disk; remote backends (S3, C
Blue/Green deployment	An RDS feature that clones the database, applies changes to the clone, and switches the endpoint aton
Data source	A read-only Terraform block that looks up an existing resource without managing it.
Drift	Divergence between the Terraform state and the real-world infrastructure, usually caused by out-of-ban
HCL	HashiCorp Configuration Language. Terraform's declarative syntax.
IAM authentication	Connecting to PostgreSQL using cloud IAM credentials in lieu of a password. Supported on RDS, Clou
Logical replication	Row-level replication between PostgreSQL instances; used for cross-version upgrades and blue/green
Module	A reusable directory of Terraform configuration with documented inputs and outputs.
Patroni	An HA orchestration system for self-hosted PostgreSQL that uses etcd, Consul, or ZooKeeper for leade
PgBouncer	A transaction-mode connection pooler for PostgreSQL.
pgaudit	A PostgreSQL extension that logs DDL, role, and write operations for audit purposes.
Plan	The output of <code>terraform plan</code> : a description of the changes Terraform propos
Provider	A Terraform plug-in that knows how to manage resources of a particular cloud or service.
PSA / PSC	Private Service Access / Private Service Connect. Two mechanisms for private networking to Google C
State	Terraform's record of which real-world resources correspond to which declared resources.
Synchronous replication	A PostgreSQL replication mode where the primary waits for the standby to confirm before acknowledg
Workspace	A namespacing mechanism inside a Terraform backend; lightweight, not a substitute for environment is

Appendix B · Command Cheat Sheet

Command	Purpose
<code>terraform init</code>	Initialise providers and backend.
<code>terraform init -reconfigure</code>	Re-initialise after a backend change.
<code>terraform init -migrate-state</code>	Move state to a new backend.
<code>terraform fmt -recursive</code>	Canonical formatting.
<code>terraform validate</code>	Syntactic checks.
<code>terraform plan -out=tfplan</code>	Produce a reviewable plan.
<code>terraform show -no-color tfplan</code>	Render the plan to text.
<code>terraform apply tfplan</code>	Apply a saved plan.
<code>terraform apply -refresh-only</code>	Update state without making changes.
<code>terraform import <addr> <id></code>	Adopt an existing resource.
<code>terraform state list</code>	List managed resource addresses.
<code>terraform state show <addr></code>	Inspect one resource's state.
<code>terraform state mv <from> <to></code>	Relocate state address.
<code>terraform state rm <addr></code>	Forget a resource.
<code>terraform taint <addr></code> (deprecated, use <code>terraform apply -replace=<addr></code>)	Replace one resource.
<code>terraform apply -replace=<addr></code>	Replace one resource.
<code>terraform workspace new <name></code>	Create a workspace.
<code>terraform output -json</code>	Emit outputs for downstream tooling.
<code>terraform providers</code>	List provider requirements.
<code>terraform graph dot -Tsvg > g.svg</code>	Render the dependency graph.

Index

Curated back-of-book index. References point to chapters and sections rather than absolute page numbers; use the running header on each body page to locate the chapter.

Term	Where to find it
AlloyDB	Ch. 6.5, Project entries
Atlas (schema migrator)	Ch. 9, Project 5
Aurora PostgreSQL	Ch. 5.2, Project 1
Azure Database for PostgreSQL	Ch. 7
Backups, pgBackRest	Ch. 4.6, Project 2
Blue/green deployment	Ch. 5.5, Glossary
CI/CD	Ch. 8.4, Project 5
Cloud SQL	Ch. 6.1–6.4, Project 3
Cloud SQL Auth Proxy	Ch. 6.3
cloud-init bootstrap	Ch. 4.4, Project 2
Compliance tagging	Ch. 9.6
Configuration drift	Ch. 1.1, Ch. 8.3
Connection pooling (PgBouncer, RDS Proxy)	Ch. 4.7, Ch. 5.3, Project 1
Cost optimisation	Ch. 6.5 (callout), Ch. 9.7
cyrilgdn/postgresql provider	Ch. 3
Default privileges	Ch. 3.4, Project 4
Drift detection	Ch. 8.3, Ch. 9
Encryption at rest (KMS / CMK)	Ch. 9.3, Project 1
Entra ID authentication (Azure AD)	Ch. 7.3, Project 4
etcd cluster	Ch. 4.5, Project 2
for_each vs count	Ch. 2.3
GitHub Actions pipeline	Ch. 8.4, Project 5
Grants and privileges	Ch. 3.4
HCL syntax	Ch. 2.1
High availability (HA)	Ch. 4 (Patroni), Ch. 5 (Multi-AZ), Ch. 6 (REGIONAL), Ch. 7 (ZoneRedundant),
IAM authentication	Ch. 5.4 (RDS), Ch. 6.4 (Cloud SQL), Ch. 7.3 (Azure)
Infrastructure as Code	Ch. 1
KMS keys	Ch. 9.3, Project 1
Lifecycle: prevent_destroy / ignore_changes	Ch. 2.2, Ch. 4.3, Ch. 7.1, Ch. 8
Logical replication	Glossary, Ch. 5.5
Module structure	Ch. 2.5, Project 1–2
moved / removed blocks	Ch. 8.2
Multi-AZ (RDS)	Ch. 5.1
Network surface minimisation	Ch. 9.2
OIDC for CI	Ch. 8.4 (callout)
Parameter groups	Ch. 5.1, Ch. 7.4
Patroni	Ch. 4.5, Project 2
Performance Insights	Ch. 5.1, Project 1
pgaudit	Ch. 9.4, every cloud chapter
PgBouncer	Ch. 4.7

Term	Where to find it
Plan / apply lifecycle	Ch. 2.7, Ch. 8
Point-in-time recovery (PITR)	Ch. 6.1, Project 3
PostgreSQL DBA role	Ch. 1
Privileges (postgresql_grant)	Ch. 3.4, Project 4
Provider, Terraform	Ch. 2.2, Ch. 3.1
Private Service Access (PSA) / Connect (PSC)	Ch. 6.2
RBAC, four-role model	Project 4
RDS PostgreSQL	Ch. 5.1, Project 1
RDS Proxy	Ch. 5.3, Project 1
Read replicas	Ch. 5.2 (Aurora), Ch. 6.5 (AlloyDB read pool), Ch. 7.5 (Azure)
Remote state backends	Ch. 2.4, Ch. 8.1
Roles and password rotation	Ch. 3.3, Project 1, Project 4
Schema migrations	Ch. 1.2, Project 5
Secret management	Ch. 9.1, Project 1, Project 4
Self-hosted PostgreSQL	Ch. 4, Project 2
Spacelift / Atlantis / TF Cloud	Ch. 8.5
State file	Ch. 2.4, Ch. 8.1–8.2
State surgery (mv / rm / import)	Ch. 8.2
Synchronous replication	Ch. 4.5, Glossary
Terraform fundamentals	Ch. 2
Variables, locals, outputs	Ch. 2.3
Workspaces	Ch. 2.6